# MÔ HÌNH HỒI QUY TUYẾN TÍNH

**Mục tiêu bài học.** Sau bài này, người học có thể:

- mô tả bài toán hồi quy bằng ký hiệu toán học và biểu diễn mô hình bằng vector, ma trận;
- hiểu hàm mất mát MSE, gradient của nó và huấn luyện mô hình bằng Gradient Descent hoặc phương trình chuẩn;
- mở rộng mô hình cho quan hệ phi tuyến bằng biến đổi đặc trưng, và nhận biết hiện tượng quá khớp;
- phân biệt MSE, MAE, Ridge, Lasso và cài đặt, đánh giá mô hình bằng Python.

**Lộ trình bài học.** Bài giảng đi theo một mạch xuyên suốt: *bài toán → mô hình → huấn luyện → mở rộng → cài đặt*.

| Mục | Nội dung | Câu hỏi chính |
|---|---|---|
| 1 | Từ bài toán dự đoán đến mô hình tuyến tính | Ta cần dự đoán gì, và vì sao dùng hàm tuyến tính? |
| 2 | Mô hình hồi quy tuyến tính | Mô hình tính giá trị dự đoán như thế nào? |
| 3 | Huấn luyện mô hình | Làm sao tìm được các tham số **w** từ dữ liệu? |
| 4 | Mở rộng mô hình bằng biến đổi đặc trưng | Nếu quan hệ không phải đường thẳng thì sao? |
| 5 | Mở rộng hàm mất mát và điều chuẩn | Làm sao kiểm soát ngoại lệ và độ phức tạp của mô hình? |
| 6 | Cài đặt và đánh giá | Mô hình học được tốt đến đâu trên dữ liệu mới? |

## 1. Từ bài toán dự đoán đến mô hình tuyến tính

Trong thực tế, ta thường cần **dự đoán một đại lượng số** dựa trên những thông tin đã biết: giá một căn nhà dựa vào diện tích và vị trí, doanh thu dựa vào chi phí quảng cáo, điểm thi dựa vào số giờ học, mức tiêu thụ điện dựa vào nhiệt độ. Đây là **bài toán hồi quy (regression)**.

Mục tiêu của bài này là xây dựng **mô hình hồi quy tuyến tính**, mô hình dự đoán số đơn giản và quan trọng nhất của học máy. Nó cũng là nền tảng để học Perceptron, Logistic Regression và mạng nơ-ron ở các bài sau. Trong mục 1, ta trả lời ba câu hỏi:

- Bài toán hồi quy được mô tả toán học như thế nào?
- Vì sao ta xấp xỉ quan hệ giữa đầu vào và đầu ra bằng một hàm tuyến tính?
- Có vô số đường thẳng có thể xấp xỉ dữ liệu, vậy làm sao chọn được một đường tốt?

## Quy ước ký hiệu

Để thống nhất với toàn bộ bài giảng, ta quy ước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng được viết dưới dạng **vector hàng**, nhờ đó các vector đặc trưng của $n$ mẫu có thể xếp chồng thành các hàng của ma trận đặc trưng.

| Đối tượng | Ký hiệu | Kích thước | Ý nghĩa |
|---|---|---|---|
| Số mẫu, số đặc trưng | $n$, $d$ | vô hướng | $n$ quan sát, mỗi quan sát có $d$ đặc trưng |
| Vector đặc trưng (hàng) | $\mathbf{x}_i$ | $1 \times (d+1)$ | Đặc trưng của mẫu thứ $i$, phần tử đầu tiên bằng 1 |
| Ma trận đặc trưng | $\mathbf{X}$ | $n \times (d+1)$ | Hàng thứ $i$ của $\mathbf{X}$ là $\mathbf{x}_i$ |
| Vector tham số (cột) | $\mathbf{w}$ | $(d+1) \times 1$ | Gồm hệ số chệch $w_0$ và các trọng số $w_1, \dots, w_d$ |
| Vector giá trị thực (cột) | $\mathbf{y}$ | $n \times 1$ | Giá trị thực tế $y_i \in \mathbb{R}$ của từng mẫu |
| Giá trị dự đoán | $\hat{y}_i$, $\hat{\mathbf{y}}$ | vô hướng, $n \times 1$ | $\hat{y}_i = \mathbf{x}_i\mathbf{w}$ và $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ |
| Sai số dự đoán | $e_i$, $\mathbf{e}$ | vô hướng, $n \times 1$ | $e_i = \hat{y}_i - y_i$ và $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$ |
| Hàm mất mát | $\ell_i(\mathbf{w})$, $J(\mathbf{w})$ | vô hướng | Mất mát của một mẫu và mất mát trên toàn bộ dữ liệu |
| Gradient | $\nabla J(\mathbf{w})$ | $(d+1) \times 1$ | Vector các đạo hàm riêng của $J$ theo từng tham số |
| Tốc độ học, siêu tham số điều chuẩn | $\eta$, $\lambda$ | vô hướng | $\eta$ dùng ở mục 3, $\lambda$ dùng ở mục 5 |

Cụ thể, với mẫu thứ $i$ ta có vector hàng và ma trận đặc trưng:

$$\mathbf{x}_i = [1 \quad x_{i1} \quad x_{i2} \quad \cdots \quad x_{id}], \qquad \mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix}$$

Vector tham số và vector giá trị thực là các vector cột:

$$\mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T, \qquad \mathbf{y} = [y_1 \quad y_2 \quad \cdots \quad y_n]^T$$

> **Lưu ý về cách viết.** Nhiều tài liệu viết vector đặc trưng là vector cột và giá trị dự đoán là $\mathbf{w}^T\mathbf{x}$. Bài giảng này viết vector đặc trưng là vector hàng nên giá trị dự đoán là $\mathbf{x}\mathbf{w}$. Hai cách viết cho cùng một giá trị, vì $\mathbf{x}\mathbf{w} = \mathbf{w}^T\mathbf{x}^T$. Với toàn bộ dữ liệu, công thức $\mathbf{X}\mathbf{w}$ không thay đổi. Cách viết vector hàng cũng được dùng nguyên vẹn ở bài Perceptron.

## 1.1. Bài toán hồi quy và quy ước dữ liệu

### Hồi quy và phân loại khác nhau ở đâu?

Hai loại bài toán học có giám sát này khác nhau ở **loại đầu ra** mà ta cần dự đoán:

- **Hồi quy** trả lời câu hỏi *"bao nhiêu?"*: giá một căn nhà, nhiệt độ ngày mai, điểm thi. Đầu ra là một **số thực**.
- **Phân loại** trả lời câu hỏi *"thuộc nhóm nào?"*: thư rác hay thư thường, đậu hay rớt. Đầu ra là một **nhãn** thuộc tập hữu hạn. Ta sẽ học bài toán này ở bài Perceptron.

### Mô tả bài toán

Giả sử ta có bộ dữ liệu huấn luyện gồm $n$ quan sát:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}, \qquad y_i \in \mathbb{R}$$

Trong đó $\mathbf{x}_i$ là vector đặc trưng (hàng) của quan sát thứ $i$, còn $y_i$ là giá trị thực tế của đại lượng cần dự đoán. Các đại lượng này có tên gọi:

- $x_1, \dots, x_d$ là **đặc trưng (feature)** hay biến đầu vào;
- $y$ là **biến mục tiêu (target)** hay biến đầu ra. Khác với phân loại, $y_i$ có thể nhận **mọi giá trị thực**.

Bài toán hồi quy là: **tìm một quy tắc dự đoán $\hat{y}$ từ đặc trưng x sao cho $\hat{y}$ gần với $y$ nhất có thể, không chỉ trên dữ liệu đã có mà còn trên dữ liệu mới.**

### Ví dụ xuyên suốt: dự đoán giá nhà theo diện tích

Ta xét một ví dụ nhỏ sẽ được dùng lại trong suốt bài giảng. Mỗi căn nhà được mô tả bởi một đặc trưng $x$ là diện tích, tính theo đơn vị 10 $m^2$, và giá bán $y$ tính theo tỷ đồng.

| Căn nhà | $x$ (diện tích, đơn vị 10 $m^2$) | $y$ (giá, tỷ đồng) |
|---|---|---|
| 1 | 3 | 1,2 |
| 2 | 4 | 1,9 |
| 3 | 5 | 2,4 |
| 4 | 6 | 3,2 |
| 5 | 7 | 3,3 |

Ở đây $n = 5$ và $d = 1$. Thêm cột 1 vào đặc trưng, ta tổ chức dữ liệu thành ma trận đặc trưng và vector giá trị thực:

$$\mathbf{X} = \begin{bmatrix} 1 & 3 \\ 1 & 4 \\ 1 & 5 \\ 1 & 6 \\ 1 & 7 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1{,}2 \\ 1{,}9 \\ 2{,}4 \\ 3{,}2 \\ 3{,}3 \end{bmatrix}$$

Mỗi hàng của **X** là vector đặc trưng của một căn nhà, ví dụ $\mathbf{x}_4 = [1 \quad 6]$ ứng với căn nhà 60 $m^2$ có giá $y_4 = 3{,}2$ tỷ đồng.

## 1.2. Ý tưởng: xấp xỉ quan hệ bằng một hàm tuyến tính

### Nhìn dữ liệu dưới dạng các điểm

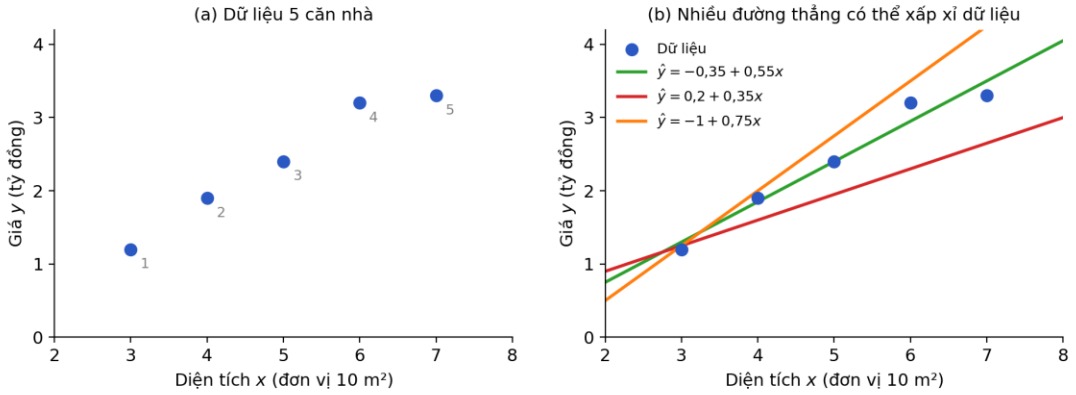
Mỗi căn nhà là một **điểm** $(x, y)$ trên mặt phẳng. Hình 1a biểu diễn 5 căn nhà trong ví dụ. Quan sát hình, ta thấy giá nhà **tăng gần như đều** khi diện tích tăng, và các điểm nằm xấp xỉ dọc theo một đường thẳng. Đây chính là ý tưởng cốt lõi của hồi quy tuyến tính:

> **Giả định rằng quan hệ giữa đầu vào và đầu ra có thể được xấp xỉ bởi một hàm tuyến tính của các đặc trưng.**

Với một đặc trưng, hàm tuyến tính là một đường thẳng:

$$\hat{y} = w_0 + w_1 x$$

Với $d$ đặc trưng, ta có $\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d$, và mục 2 sẽ viết nó thật gọn bằng vector và ma trận.




*Hình 1. (a) Dữ liệu 5 căn nhà. (b) Nhiều đường thẳng khác nhau cùng xấp xỉ dữ liệu: đường xanh lá (A), đường đỏ (B), đường cam (C).*

### Vì sao chọn hàm tuyến tính?

Việc chọn hàm tuyến tính có ba lý do:

- **Đơn giản:** chỉ cần $d + 1$ tham số, dễ huấn luyện và dễ cài đặt.
- **Dễ diễn giải:** mỗi tham số cho biết ảnh hưởng của một đặc trưng đến giá trị dự đoán (mục 2.1).
- **Là nền tảng:** các mô hình phức tạp hơn, từ Perceptron đến mạng nơ-ron, đều xây dựng từ biểu thức tuyến tính này. Hơn nữa, mục 4 sẽ chỉ ra rằng chỉ cần biến đổi đặc trưng, mô hình tuyến tính vẫn biểu diễn được quan hệ cong.

## Có rất nhiều đường thẳng: làm sao chọn một đường?

Mỗi bộ tham số $(w_0, w_1)$ cho một đường thẳng khác nhau. Hình 1b vẽ ba đường:

- đường A: $\hat{y} = -0{,}35 + 0{,}55x$;
- đường B: $\hat{y} = 0{,}2 + 0{,}35x$;
- đường C: $\hat{y} = -1 + 0{,}75x$.

Bảng sau cho giá trị dự đoán của ba đường trên 5 căn nhà:

| Căn nhà | $y$ | A: $\hat{y}$ | B: $\hat{y}$ | C: $\hat{y}$ |
|---|---|---|---|---|
| 1 | 1,2 | 1,30 | 1,25 | 1,25 |
| 2 | 1,9 | 1,85 | 1,60 | 2,00 |
| 3 | 2,4 | 2,40 | 1,95 | 2,75 |
| 4 | 3,2 | 2,95 | 2,30 | 3,50 |
| 5 | 3,3 | 3,50 | 2,65 | 4,25 |
| **Sai số bình phương trung bình** | | **0,023** | **0,3055** | **0,2255** |

Dòng cuối là trung bình của bình phương độ lệch giữa $\hat{y}$ và $y$, đại lượng sẽ được định nghĩa chính thức ở mục 3.2 với tên **MSE**. Đường A có sai số nhỏ hơn hẳn hai đường còn lại. Điều đó khớp với cảm nhận bằng mắt ở Hình 1b, nhưng ta không thể dựa vào cảm nhận: với nhiều đặc trưng ta không thể vẽ ra hình, và máy tính cần một **tiêu chí định lượng** để tự chọn đường tốt nhất.

## Khung huấn luyện: mô hình, hàm mất mát và thuật toán tối ưu

Tình huống trên dẫn đến câu hỏi trung tâm của bài giảng:

**Các tham số w lấy ở đâu?**

Câu trả lời là một **quá trình huấn luyện** để máy tính tự tìm **w** từ dữ liệu. Quá trình này luôn gồm ba thành phần, và đây là khung tư duy đúng với hầu hết mô hình học máy:

$$\text{Mô hình} + \text{Hàm mất mát} + \text{Thuật toán tối ưu} = \text{Huấn luyện}$$

| Thành phần | Trả lời câu hỏi | Trong bài hồi quy tuyến tính |
|---|---|---|
| **Mô hình** | Ta dùng dạng hàm nào để mô tả quan hệ giữa đầu vào và đầu ra? | $\hat{y} = \mathbf{xw}$ (mục 2) |
| **Hàm mất mát** | Bộ tham số hiện tại tốt hay kém trên dữ liệu? | MSE (mục 3.2) |
| **Thuật toán tối ưu** | Chỉnh tham số như thế nào để mô hình tốt hơn? | Gradient Descent, phương trình chuẩn (mục 3.3 đến 3.7) |

Các mục 4 và 5 sẽ cho thấy ta có thể thay đổi từng thành phần riêng lẻ: đổi cách biểu diễn đặc trưng (mục 4), đổi hàm mất mát (mục 5).

## Tóm tắt mục 1

- **Hồi quy** dự đoán một đại lượng số thực $y \in \mathbb{R}$, khác với phân loại dự đoán một nhãn.
- Dữ liệu được tổ chức thành ma trận đặc trưng $\mathbf{X}$ (mỗi hàng là một vector hàng $\mathbf{x}_i$ có phần tử đầu bằng 1) và vector giá trị thực $\mathbf{y}$.
- Ý tưởng hồi quy tuyến tính: **xấp xỉ quan hệ giữa đầu vào và đầu ra bằng một hàm tuyến tính** $\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d$.
- Có vô số đường thẳng như vậy, nên cần một **thước đo sai số** và một **quá trình huấn luyện** để tìm bộ tham số $\mathbf{w}$ tốt nhất.
- Huấn luyện luôn gồm ba thành phần: **mô hình, hàm mất mát, thuật toán tối ưu**.

## Câu hỏi kiểm tra nhanh

1. Với dữ liệu 5 căn nhà, hãy viết $\mathbf{x}_3$ và $y_3$. Ma trận $\mathbf{X}$ có kích thước bao nhiêu?
2. Trong các bài toán sau, bài nào là hồi quy, bài nào là phân loại? (a) Dự đoán một email có phải thư rác không. (b) Dự đoán nhiệt độ ngày mai. (c) Dự đoán số phút trễ của một chuyến bay. (d) Dự đoán khách hàng có rời bỏ dịch vụ hay không.
3. Dùng đường B: $\hat{y} = 0{,}2 + 0{,}35x$ để dự đoán giá căn nhà $50\ m^2$ ($x = 5$). Giá dự đoán chênh lệch bao nhiêu so với giá thực $y_3 = 2{,}4$?
4. Một bộ dữ liệu có $n = 100$ mẫu và $d = 3$ đặc trưng. Hãy cho biết kích thước của $\mathbf{x}_i$, $\mathbf{X}$ và $\mathbf{w}$.
5. Vì sao ta cần một thước đo sai số thay vì chọn đường thẳng bằng cách nhìn đồ thị?

### Gợi ý đáp án

1. $\mathbf{x}_3 = [1 \quad 5]$ và $y_3 = 2{,}4$. Ma trận $\mathbf{X}$ có kích thước $5 \times 2$.
2. 
   (b) và (c) là hồi quy vì đầu ra là số thực. (a) và (d) là phân loại vì đầu ra là nhãn.
3. $\hat{y} = 0{,}2 + 0{,}35 \cdot 5 = 1{,}95$ tỷ đồng. Chênh lệch $\hat{y} - y_3 = 1{,}95 - 2{,}4 = -0{,}45$, nghĩa là dự đoán thấp hơn thực tế 0,45 tỷ đồng.
4. $\mathbf{x}_i$ có kích thước $1 \times 4$, $\mathbf{X}$ có kích thước $100 \times 4$, $\mathbf{w}$ có kích thước $4 \times 1$ (do có thêm hệ số chệch $w_0$ và cột 1).
5. Có vô số đường thẳng xấp xỉ dữ liệu. Muốn so sánh chúng và để máy tính tự tìm đường tốt nhất, ta cần một đại lượng **định lượng** đo mức sai lệch. Với nhiều đặc trưng ta cũng không thể vẽ đồ thị để nhìn.

# 2. Mô hình hồi quy tuyến tính

Ở mục 1, ta đã thấy ý tưởng xấp xỉ quan hệ bằng một hàm tuyến tính. Mục này biến ý tưởng đó thành một mô hình hoàn chỉnh. Ta trả lời ba câu hỏi:

- Mô hình nhận đầu vào và tính giá trị dự đoán như thế nào, và các tham số có ý nghĩa gì?
- Mô hình được viết gọn bằng vector và ma trận ra sao, và sai số dự đoán được tính thế nào?
- Về mặt hình học, mô hình tạo ra đường (mặt) nào, và các tham số điều khiển nó ra sao?

Ta tiếp tục dùng quy ước ký hiệu ở mục 1: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng** với phần tử đầu tiên bằng 1, vector tham số $\mathbf{w}$ là **vector cột**.

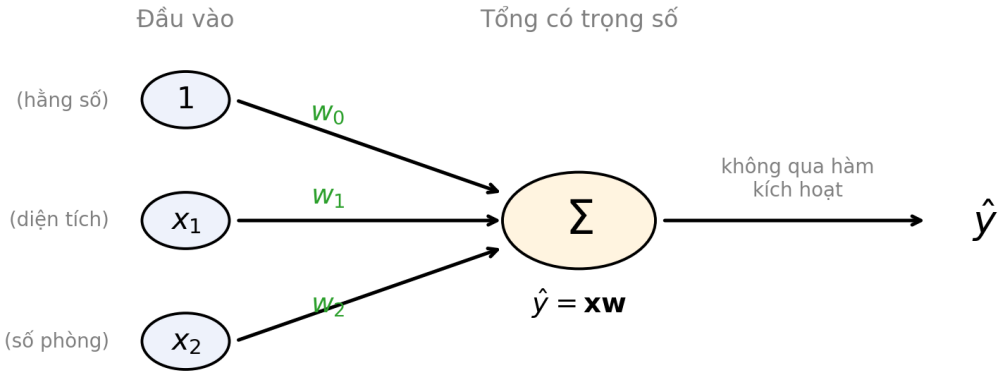
## 2.1. Tổng có trọng số và hệ số chệch

### Ý tưởng hoạt động: một "máy dự đoán" hai bước

Có thể hình dung mô hình như một người ước lượng giá nhà bằng cách **cân nhắc nhiều yếu tố**, mỗi yếu tố có một mức độ quan trọng riêng. Quy trình gồm hai bước:

1. **Nhận đặc trưng.** Mô hình nhận vào các đặc trưng $x_1, x_2, \ldots, x_d$ của một mẫu.
2. **Tính tổng có trọng số.** Mỗi đặc trưng được nhân với một **trọng số** thể hiện mức độ quan trọng, rồi cộng tất cả lại và cộng thêm một hằng số điều chỉnh. Kết quả chính là giá trị dự đoán $\hat{y}$.

Hình 2 minh họa quy trình này với hai đặc trưng. Khác với Perceptron ở bài sau, mô hình hồi quy **không có bước thứ ba**: giá trị tổng có trọng số được dùng trực tiếp làm đầu ra.




*Hình 2. Cấu trúc của mô hình hồi quy tuyến tính với hai đặc trưng. Các đầu vào được nhân với trọng số rồi cộng lại thành giá trị dự đoán ŷ, không qua hàm kích hoạt.*

### Hàm tổng hợp có trọng số và hệ số chệch

Với một mẫu có các đặc trưng $x_1, \ldots, x_d$, mô hình dự đoán

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = w_0 + \sum_{j=1}^{d} w_j \, x_j$$

Trong đó:

- $w_1, \ldots, w_d$ là các **trọng số (weight)**. Trọng số $w_j$ cho biết khi $x_j$ **tăng một đơn vị** và các đặc trưng khác giữ cố định thì giá trị dự đoán $\hat{y}$ thay đổi $w_j$ đơn vị. Nếu $w_j > 0$ thì $x_j$

tăng làm $\hat{y}$ tăng, nếu $w_j < 0$ thì $x_j$ tăng làm $\hat{y}$ giảm, và nếu $w_j = 0$ thì đặc trưng $x_j$ không ảnh hưởng đến dự đoán.

- $w_0$ là **hệ số chệch (bias, intercept)**. Nó là giá trị dự đoán khi mọi đặc trưng bằng 0, và đóng vai trò như một "mức khởi điểm" của mô hình.

**Ví dụ với hai đặc trưng.** Giả sử $x_1$ là diện tích ($m^2$), $x_2$ là khoảng cách đến trung tâm ($km$), và mô hình đã có $w_0 = 0{,}8$, $w_1 = 0{,}045$, $w_2 = -0{,}08$:

$$\hat{y} = 0{,}8 + 0{,}045\, x_1 - 0{,}08\, x_2$$

Với căn nhà 70 $m^2$ cách trung tâm 5 $km$: $\hat{y} = 0{,}8 + 0{,}045 \cdot 70 - 0{,}08 \cdot 5 = 3{,}55$ tỷ đồng. Hệ số $w_1 = 0{,}045$ cho biết mỗi $m^2$ tăng thêm làm giá dự đoán tăng 0,045 tỷ đồng (45 triệu), còn $w_2 = -0{,}08$ cho biết mỗi $km$ xa trung tâm hơn làm giá dự đoán giảm 0,08 tỷ đồng.

**Hai lưu ý khi diễn giải tham số.**

(1) Độ lớn của các trọng số **không** dùng trực tiếp để so sánh mức độ quan trọng của các đặc trưng nếu chúng đo trên các thang khác nhau. Ở ví dụ trên, không thể kết luận khoảng cách "quan trọng hơn" diện tích chỉ vì $|w_2| > |w_1|$, vì một đặc trưng tính bằng $m^2$ còn đặc trưng kia tính bằng $km$.

(2) Hệ số chệch $w_0$ chỉ có ý nghĩa thực tế khi giá trị 0 của các đặc trưng nằm trong phạm vi dữ liệu. Ở ví dụ 5 căn nhà, đường A cho $\hat{y} = -0{,}35$ khi $x = 0$, một giá âm vô nghĩa. Mô hình chỉ đáng tin trong khoảng diện tích đã quan sát (30 đến 70 $m^2$).

### Dạng vector

Nhờ thêm số 1 vào đầu vector đặc trưng, $\mathbf{x} = [1 \quad x_1 \quad \cdots \quad x_d]$, toàn bộ biểu thức trên được viết gọn thành

$$\hat{y} = \mathbf{x}\mathbf{w}, \qquad \mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T$$

Số 1 ở đầu $\mathbf{x}$ được nhân với $w_0$, nhờ vậy hệ số chệch được gộp vào cùng một phép nhân với các trọng số khác. Đây là thủ thuật giúp mọi công thức phía sau gọn hơn.

## 2.2. Biểu diễn vector và ma trận của mô hình

### Một mẫu

Với mẫu thứ $i$, giá trị dự đoán là tích của vector hàng $\mathbf{x}_i$ (kích thước $1 \times (d+1)$) và vector cột $\mathbf{w}$ (kích thước $(d+1) \times 1$), cho ra một số:

$$\hat{y}_i = \mathbf{x}_i\mathbf{w} = w_0 + w_1 x_{i1} + \cdots + w_d x_{id}$$

### Toàn bộ $n$ mẫu cùng lúc

Thay vì tính từng dự đoán $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n$, ta xếp chúng thành vector cột $\hat{\mathbf{y}}$ và tính **đồng thời** bằng một phép nhân ma trận:

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1\mathbf{w} \\ \mathbf{x}_2\mathbf{w} \\ \vdots \\ \mathbf{x}_n\mathbf{w} \end{bmatrix} = \mathbf{X}\mathbf{w}$$

Về kích thước, $\mathbf{X}$ là $n \times (d+1)$ và $\mathbf{w}$ là $(d+1) \times 1$ nên $\mathbf{X}\mathbf{w}$ là $n \times 1$, khớp với $\hat{\mathbf{y}}$. Đây là một trong những lý do biểu diễn ma trận rất quan trọng trong học máy: một dòng mã, một phép nhân ma trận, cho toàn bộ dự đoán.

## Sai số dự đoán

Giá trị dự đoán thường không trùng với giá trị thực tế. Ta gọi hiệu của hai giá trị là **sai số dự đoán (error, residual)**:

$$e_i = \hat{y}_i - y_i, \qquad \mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = \mathbf{X}\mathbf{w} - \mathbf{y}$$

Dấu của $e_i$ cho biết hướng sai: $e_i > 0$ nghĩa là mô hình **dự đoán cao hơn** thực tế, $e_i < 0$ nghĩa là **dự đoán thấp hơn**, và $e_i = 0$ là dự đoán chính xác. Vector $\mathbf{e}$ là điểm xuất phát để xây dựng hàm mất mát ở mục 3.

## Ví dụ số với dữ liệu nhà

Chọn vector tham số của đường A:

$$\mathbf{w} = \begin{bmatrix} -0{,}35 \\ 0{,}55 \end{bmatrix}$$

Giá trị dự đoán và sai số của cả 5 căn nhà được tính đồng thời:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & 3 \\ 1 & 4 \\ 1 & 5 \\ 1 & 6 \\ 1 & 7 \end{bmatrix} \begin{bmatrix} -0{,}35 \\ 0{,}55 \end{bmatrix} = \begin{bmatrix} 1{,}30 \\ 1{,}85 \\ 2{,}40 \\ 2{,}95 \\ 3{,}50 \end{bmatrix}, \qquad \mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 0{,}10 \\ -0{,}05 \\ 0 \\ -0{,}25 \\ 0{,}20 \end{bmatrix}$$

| Căn nhà | $\mathbf{x}_i$ | $y_i$ | $\hat{y}_i$ | $e_i = \hat{y}_i - y_i$ | Nhận xét |
|---|---|---|---|---|---|
| 1 | $[1 \quad 3]$ | 1,2 | 1,30 | 0,10 | Dự đoán cao hơn |
| 2 | $[1 \quad 4]$ | 1,9 | 1,85 | −0,05 | Dự đoán thấp hơn |
| 3 | $[1 \quad 5]$ | 2,4 | 2,40 | 0 | Chính xác |
| 4 | $[1 \quad 6]$ | 3,2 | 2,95 | −0,25 | Dự đoán thấp hơn |
| 5 | $[1 \quad 7]$ | 3,3 | 3,50 | 0,20 | Dự đoán cao hơn |

Các sai số đều nhỏ, và một số dương, một số âm. Điều này cho thấy đường A bám sát dữ liệu.

## 2.3. Hình học của mô hình hồi quy

### Đường hồi quy và siêu phẳng hồi quy

Tập hợp các điểm $(\mathbf{x}, \hat{y})$ thỏa $\hat{y} = \mathbf{xw}$ có hình dạng phụ thuộc vào số đặc trưng:

- Khi $d = 1$, đó là một **đường thẳng** trong mặt phẳng $(x, y)$, như đường A ở Hình 1b.
- Khi $d = 2$, đó là một **mặt phẳng** trong không gian ba chiều $(x_1, x_2, y)$.
- Khi $d > 2$, ta không thể vẽ ra nhưng vẫn gọi là một **siêu phẳng (hyperplane)** trong không gian $d + 1$ chiều.

**So sánh với Perceptron.** Biểu thức **xw** cũng xuất hiện ở Perceptron, nhưng ý nghĩa hình học khác. Ở Perceptron, siêu phẳng $\mathbf{xw} = 0$ nằm **trong không gian đặc trưng** và đóng vai trò **đường biên** chia hai lớp. Ở hồi quy, siêu phẳng $y = \mathbf{xw}$ nằm **trong không gian đặc trưng cộng thêm trục đầu ra**, và đóng vai trò **bề mặt dự đoán** đi xuyên qua đám dữ liệu.

### Vai trò của các tham số

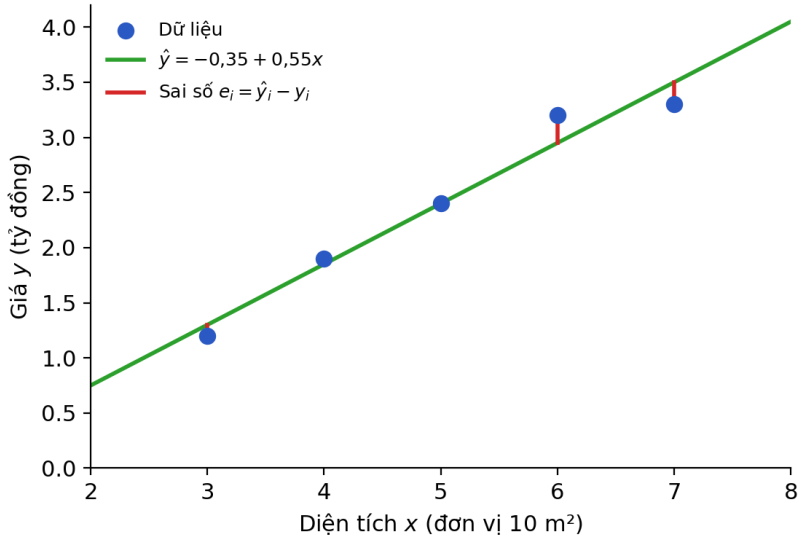
Với $d = 1$, hai tham số có ý nghĩa hình học rõ ràng:

- $w_0$ là **tung độ gốc**: thay đổi $w_0$ làm đường thẳng **tịnh tiến lên xuống** mà không đổi hướng.
- $w_1$ là **hệ số góc**: thay đổi $w_1$ làm đường thẳng **xoay** quanh một điểm. Ở đường A, $w_1 = 0{,}55$ nghĩa là thêm 10 $m^2$ diện tích làm giá dự đoán tăng 0,55 tỷ đồng.

Ba đường ở Hình 1b khác nhau chính là do hai tham số này: đường B có hệ số góc nhỏ hơn nên thoải hơn dữ liệu, đường C có hệ số góc lớn hơn nên dốc hơn dữ liệu.

### Sai số được đo theo phương dọc

Sai số $e_i$ ứng với **khoảng cách theo phương thẳng đứng** (phương trục $y$) từ điểm dữ liệu đến đường hồi quy, như các đoạn đỏ ở Hình 3. Đây **không** phải khoảng cách vuông góc từ điểm đến đường thẳng. Lý do là ta quan tâm đến việc dự đoán $y$ sai bao nhiêu khi đã biết $x$.




*Hình 3. Đường hồi quy $\hat{y} = -0{,}35 + 0{,}55x$ và sai số $e_i$ của từng căn nhà (đoạn màu đỏ, đo theo phương thẳng đứng).*

### Tuyến tính theo tham số

Cuối cùng, cần nhấn mạnh rằng mô hình $\hat{y} = \mathbf{xw}$ là **tuyến tính theo các tham số w**. Nếu ta nhân đôi mọi tham số thì giá trị dự đoán cũng nhân đôi, và tổng của hai bộ tham số cho dự đoán bằng tổng hai dự đoán. Tính chất này, chứ không phải tính tuyến tính theo đặc trưng, là điều làm cho việc huấn luyện ở mục 3 đơn giản. Mục 4 sẽ khai thác sự khác biệt này.

## Tóm tắt mục 2

- Mô hình hồi quy tuyến tính tính giá trị dự đoán bằng **tổng có trọng số cộng hệ số chệch**: $\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d = \mathbf{xw}$.
- Trọng số $w_j$ là mức thay đổi của $\hat{y}$ khi $x_j$ tăng một đơn vị (giữ nguyên các đặc trưng khác). Không so sánh độ lớn trọng số giữa các đặc trưng có thang đo khác nhau.
- Với toàn bộ dữ liệu, ta viết $\hat{\mathbf{y}} = \mathbf{Xw}$ và vector sai số $\mathbf{e} = \mathbf{Xw} - \mathbf{y}$. Sai số dương nghĩa là dự đoán cao hơn thực tế.
- Về hình học, mô hình tạo ra một **đường thẳng** ($d = 1$), **mặt phẳng** ($d = 2$) hoặc **siêu phẳng** ($d > 2$). Tham số $w_0$ tịnh tiến, các trọng số làm xoay.
- Mô hình **tuyến tính theo tham số w**.

## Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [0{,}8 \quad 0{,}045 \quad -0{,}08]^T$ và một căn nhà có $\mathbf{x} = [1 \quad 90 \quad 3]$, hãy tính giá dự đoán $\hat{y}$.
2. Giải thích bằng lời ý nghĩa của hệ số $w_2 = -0{,}08$ trong câu 1.
3. Với đường A và căn nhà 45 $m^2$ ($x = 4{,}5$), hãy tính $\hat{y}$.
4. Cho ba mẫu có $\mathbf{x}_1 = [1 \quad 2]$, $\mathbf{x}_2 = [1 \quad 4]$, $\mathbf{x}_3 = [1 \quad 6]$ và $\mathbf{y} = [2{,}5 \quad 3 \quad 3{,}5]^T$. Với $\mathbf{w} = [1 \quad 0{,}5]^T$, hãy tính $\hat{\mathbf{y}}$ và $\mathbf{e}$.

5. Vì sao không nên diễn giải $w_0 = -0{,}35$ của đường A là "giá của một căn nhà có diện tích bằng 0"?

### Gợi ý đáp án

1. $\hat{y} = 0{,}8 + 0{,}045 \cdot 90 - 0{,}08 \cdot 3 = 0{,}8 + 4{,}05 - 0{,}24 = 4{,}61$ tỷ đồng.
2. Khi khoảng cách đến trung tâm tăng thêm 1 $km$ và diện tích không đổi, giá dự đoán **giảm** 0,08 tỷ đồng (80 triệu đồng).
3. $\hat{y} = -0{,}35 + 0{,}55 \cdot 4{,}5 = -0{,}35 + 2{,}475 = 2{,}125$ tỷ đồng.
4. $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = [1 + 0{,}5 \cdot 2 \quad 1 + 0{,}5 \cdot 4 \quad 1 + 0{,}5 \cdot 6]^T = [2 \quad 3 \quad 4]^T$ và $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = [-0{,}5 \quad 0 \quad 0{,}5]^T$.
5. Giá âm là vô nghĩa, và diện tích bằng 0 nằm ngoài phạm vi dữ liệu (30 đến 70 $m^2$). Mô hình chỉ được huấn luyện và chỉ đáng tin trong khoảng dữ liệu đã quan sát, nên $w_0$ chủ yếu là tham số giúp đường thẳng dịch đúng vị trí.

# 3. Huấn luyện mô hình hồi quy tuyến tính

Ở mục 2, ta đã có mô hình $\hat{y} = \mathbf{x}\mathbf{w}$. Tuy nhiên, mô hình này mới chỉ là một "chiếc máy" chưa được cài đặt: vector tham số $\mathbf{w}$ vẫn chưa biết. Ta đã thấy có vô số $\mathbf{w}$ khác nhau, và chúng cho các đường thẳng tốt xấu khác nhau. Làm sao để máy tính **tự tìm ra** bộ $\mathbf{w}$ tốt nhất từ dữ liệu? Đó là bài toán **huấn luyện (training)**.

Trong mục này, ta trả lời ba câu hỏi, bám theo khung *mô hình, hàm mất mát, thuật toán tối ưu* ở mục 1:

- Làm thế nào để **đo mức độ sai** của một bộ tham số? (hàm mất mát)
- Với một bộ tham số hiện tại, **nên chỉnh w theo hướng nào** để sai số giảm? (gradient và quy tắc cập nhật)
- Lặp lại việc chỉnh sửa ra sao, **khi nào thì dừng**, và có cách tìm $\mathbf{w}$ **trực tiếp** không? (thuật toán)

Ta tiếp tục dùng quy ước ký hiệu ở các mục trước: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, giá trị dự đoán $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ và sai số $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$.

## 3.1. Mục tiêu huấn luyện và vòng lặp học

Mỗi lựa chọn $\mathbf{w}$ tạo ra một hàm dự đoán khác nhau và do đó một mức sai số khác nhau. Vì vậy, bài toán huấn luyện được phát biểu như sau:

> **Tìm bộ tham số w sao cho hàm mất mát trên dữ liệu huấn luyện đạt giá trị nhỏ nhất.**

Về mặt toán học:

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}}\, J(\mathbf{w})$$

Toàn bộ quá trình có thể mô tả bằng chuỗi:

$$\mathcal{D} \to \mathbf{X}, \mathbf{y} \to \hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \to J(\mathbf{w}) \to \mathbf{w}^*$$

Nếu giải bài toán bằng cách **cải thiện dần**, ta có một vòng lặp mà mô hình "học" qua từng vòng:

$$\mathbf{w} \to \hat{\mathbf{y}} \to J(\mathbf{w}) \to \nabla J(\mathbf{w}) \to \mathbf{w}_{\text{mới}} \to \hat{\mathbf{y}} \to \cdots$$

Mô hình không "tự biết" tham số đúng ngay từ đầu. Nó bắt đầu từ một giá trị ban đầu, đo sai số, xác định hướng cần điều chỉnh và cập nhật tham số để giảm sai số. Các mục 3.2 đến 3.5 xây dựng từng mắt xích của vòng lặp này, còn mục 3.7 trình bày cách giải trực tiếp không cần lặp.

## 3.2. Hàm mất mát MSE

### Từ sai số từng mẫu đến mất mát toàn bộ

Với $n$ mẫu, mô hình tạo ra $n$ sai số $e_1, \dots, e_n$. Ta cần **một con số** tổng hợp chúng. Có hai bước:

**Mất mát trên một mẫu.** Ta dùng bình phương của sai số:

$$\ell_i(\mathbf{w}) = e_i^2 = (\hat{y}_i - y_i)^2 = (\mathbf{x}_i\mathbf{w} - y_i)^2$$

**Mất mát trên toàn bộ dữ liệu.** Ta lấy trung bình của các mất mát từng mẫu, gọi là **sai số bình phương trung bình (Mean Squared Error, MSE)**:

$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} (\mathbf{x}_i\mathbf{w} - y_i)^2$$

Giá trị $J(\mathbf{w})$ càng nhỏ thì các giá trị dự đoán càng gần giá trị thực tế trên dữ liệu huấn luyện. Việc phân biệt $\ell_i$ (một mẫu) và $J$ (toàn bộ dữ liệu) sẽ rất hữu ích ở mục 3.4, khi ta chia dữ liệu thành các nhóm nhỏ.

**Vì sao bình phương sai số?** Có ba lý do:

- **Không triệt tiêu nhau.** Sai số dương và âm sẽ cộng lại bằng 0 nếu ta chỉ lấy tổng $e_i$. Ở ví dụ đường A, tổng các sai số $0{,}10 - 0{,}05 + 0 - 0{,}25 + 0{,}20$ đúng bằng 0 dù mô hình không hoàn hảo.
- **Phạt mạnh sai số lớn.** Sai số gấp 5 lần thì bị phạt gấp 25 lần. Đây vừa là ưu điểm, vừa là hạn chế khi dữ liệu có ngoại lệ (mục 5.1).
- **Hàm trơn.** $e_i^2$ khả vi tại mọi điểm nên ta tính được gradient một cách thuận tiện.

### Dạng ma trận

Vì $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$ và $\| \mathbf{e} \|^2 = \sum_{i=1}^{n} e_i^2$ là bình phương chuẩn Euclid của $\mathbf{e}$, ta viết MSE gọn thành

$$J(\mathbf{w}) = \frac{1}{n} \| \mathbf{X}\mathbf{w} - \mathbf{y} \|^2 = \frac{1}{n}\mathbf{e}^T\mathbf{e}$$

Bài toán huấn luyện với MSE trở thành

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{n} \| \mathbf{X}\mathbf{w} - \mathbf{y} \|^2$$

### Ví dụ số với dữ liệu nhà

Với đường A ở mục 2.2, $\mathbf{e} = [0{,}10 \quad -0{,}05 \quad 0 \quad -0{,}25 \quad 0{,}20]^T$ nên

$$J(\mathbf{w}) = \frac{1}{5}(0{,}10^2 + 0{,}05^2 + 0^2 + 0{,}25^2 + 0{,}20^2) = \frac{0{,}115}{5} = 0{,}023$$

Ta có thể so sánh với các bộ tham số khác:

| Bộ tham số $\mathbf{w}$ | $J(\mathbf{w})$ | Nhận xét |
|---|---|---|
| $[-0{,}35 \quad 0{,}55]^T$ (đường A) | 0,023 | Rất nhỏ, bám sát dữ liệu |
| $[-1 \quad 0{,}75]^T$ (đường C) | 0,2255 | Dốc hơn dữ liệu |
| $[0{,}2 \quad 0{,}35]^T$ (đường B) | 0,3055 | Thoải hơn dữ liệu |
| $[0 \quad 0]^T$ (mô hình luôn dự đoán 0) | 6,388 | Rất tệ, đây là điểm khởi đầu ở mục 3.5 |

### Hình dạng của hàm mất mát

Vì $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$ là hàm bậc nhất của $\mathbf{w}$, nên $J(\mathbf{w})$ là hàm **bậc hai** của $\mathbf{w}$. Đồ thị của nó theo hai tham số $(w_0, w_1)$ có dạng **một cái bát** với **đúng một điểm thấp nhất** (khi các cột của $\mathbf{X}$ độc lập tuyến tính). Vì không có điểm cực tiểu địa phương nào khác, mọi thuật toán đi xuống đều dẫn tới cùng một đáp án $\mathbf{w}^*$. Đây là một đặc điểm rất thuận lợi của hồi quy tuyến tính. Hình 4 ở mục 3.5 vẽ các đường mức của cái bát này.

## 3.3. Gradient và quy tắc cập nhật

### Gradient cho ta biết điều gì?

Gradient của $J(\mathbf{w})$ là vector chứa các đạo hàm riêng theo từng tham số:

$$\nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial J}{\partial w_0} \\ \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_d} \end{bmatrix}$$

Gradient cho biết **nếu thay đổi các tham số theo những hướng khác nhau thì hàm mất mát thay đổi như thế nào**. Đặc biệt, gradient chỉ theo hướng mà $J$ **tăng nhanh nhất**. Do đó, hướng $-\nabla J(\mathbf{w})$ là hướng làm $J$ **giảm nhanh nhất** tại vị trí hiện tại. Đây là cơ sở của phương pháp **Gradient Descent (hạ gradient)**.

## Gradient của mất mát một mẫu

Ta bắt đầu từ một mẫu. Với $\ell_i(\mathbf{w}) = (\mathbf{x}_i\mathbf{w} - y_i)^2$, theo quy tắc đạo hàm của hàm hợp ta có

$$\nabla \ell_i(\mathbf{w}) = 2(\mathbf{x}_i\mathbf{w} - y_i)\mathbf{x}_i^T = 2e_i\,\mathbf{x}_i^T$$

Ở đây ta dùng $\mathbf{x}_i^T$ (vector cột) để gradient có cùng kích thước $(d+1)\times 1$ với $\mathbf{w}$. Viết theo từng thành phần, $\frac{\partial \ell_i}{\partial w_0} = 2e_i$ và $\frac{\partial \ell_i}{\partial w_j} = 2e_i x_{ij}$.

## Gradient của MSE

Vì $J$ là trung bình của các $\ell_i$, gradient của $J$ là trung bình các gradient:

$$\nabla J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} 2\,e_i\,\mathbf{x}_i^T = \frac{2}{n}\mathbf{X}^T\mathbf{e} = \frac{2}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

Đẳng thức giữa tổng và tích ma trận đúng vì $\mathbf{X}^T$ có các cột là $\mathbf{x}_1^T, \dots, \mathbf{x}_n^T$, nên $\mathbf{X}^T\mathbf{e} = \sum_i e_i\,\mathbf{x}_i^T$. Về kích thước, $\mathbf{X}^T$ là $(d+1)\times n$ và $\mathbf{e}$ là $n\times 1$ nên $\mathbf{X}^T\mathbf{e}$ là $(d+1)\times 1$, khớp với $\mathbf{w}$.

Viết theo từng thành phần (cột đầu của $\mathbf{X}$ toàn số 1 nên đạo hàm theo $w_0$ đơn giản hơn):

$$\frac{\partial J}{\partial w_0} = \frac{2}{n}\sum_{i=1}^{n} e_i\,, \qquad \frac{\partial J}{\partial w_j} = \frac{2}{n}\sum_{i=1}^{n} e_i\,x_{ij} \quad (j = 1, \dots, d)$$

## Quy tắc cập nhật

Áp dụng Gradient Descent, từ bộ tham số hiện tại $\mathbf{w}$ ta đi một bước theo hướng $-\nabla J$:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla J(\mathbf{w}) = \mathbf{w} - \eta\,\frac{2}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

Trong đó $\eta > 0$ là **tốc độ học (learning rate)**, quyết định **độ dài của bước đi** trong không gian tham số. Viết theo từng thành phần:

$$w_0 \leftarrow w_0 - \eta\,\frac{2}{n}\sum_{i=1}^{n} e_i\,, \qquad w_j \leftarrow w_j - \eta\,\frac{2}{n}\sum_{i=1}^{n} e_i\,x_{ij}$$

**Ý nghĩa.** Mỗi mẫu $i$ đóng góp vào bước cập nhật một lượng $e_i\mathbf{x}_i^T$, tỉ lệ với sai số của nó:

- Nếu $e_i > 0$ (dự đoán **quá cao**), ta **trừ** một phần $\mathbf{x}_i^T$ khỏi $\mathbf{w}$ để dự đoán của mẫu này giảm xuống.
- Nếu $e_i < 0$ (dự đoán **quá thấp**), ta **cộng** một phần $\mathbf{x}_i^T$ vào $\mathbf{w}$ để dự đoán của mẫu này tăng lên.
- Mẫu có sai số càng lớn thì đóng góp càng nhiều.

### Vì sao quy tắc này hợp lý?

Ta kiểm tra điều gì xảy ra với **một mẫu** nếu chỉ cập nhật theo mẫu đó: $\mathbf{w}' = \mathbf{w} - \eta\, 2e_i \mathbf{x}_i^T$. Dự đoán mới của mẫu $i$ là

$$\hat{y}_i{}' = \mathbf{x}_i \mathbf{w}' = \mathbf{x}_i \mathbf{w} - 2\eta\, e_i\, \mathbf{x}_i \mathbf{x}_i^T = \hat{y}_i - 2\eta\, e_i \parallel \mathbf{x}_i \parallel^2$$

Trừ cả hai vế cho $y_i$, ta được sai số mới

$$e_i{}' = e_i(1 - 2\eta \parallel \mathbf{x}_i \parallel^2)$$

Nếu $0 < \eta < 1/\parallel \mathbf{x}_i \parallel^2$ thì $|e_i{}'| < |e_i|$: mỗi bước cập nhật **làm giảm sai số của mẫu đang xét**. Nếu $\eta$ quá lớn thì thừa số $1 - 2\eta \parallel \mathbf{x}_i \parallel^2$ có thể nhỏ hơn $-1$, sai số **tăng** thay vì giảm. Đây là mầm mống của hiện tượng phân kỳ ở mục 3.6. Cần lưu ý rằng một bước cập nhật theo toàn bộ dữ liệu là sự cân bằng giữa mọi mẫu, nên không mẫu nào được cải thiện chắc chắn, nhưng $J$ sẽ giảm nếu $\eta$ đủ nhỏ.

### So sánh với Perceptron ở bài sau

Quy tắc cập nhật của Perceptron cũng có dạng $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot (\text{gradient})$, nhưng khác ở hai điểm:

| Tiêu chí | Hồi quy tuyến tính (MSE) | Perceptron |
|---|---|---|
| Mẫu nào đóng góp | **Mọi** mẫu | **Chỉ** mẫu bị phân loại sai |
| Mức đóng góp | Tỉ lệ với sai số liên tục $e_i$ | Luôn là $\pm\eta\, \mathbf{x}_i^T$ |
| Cách duyệt dữ liệu | Thường dùng toàn bộ dữ liệu cho một lần cập nhật | Cập nhật ngay sau mỗi mẫu |

## 3.4. Thuật toán Gradient Descent qua từng bước

### Một số khái niệm

- Một lần **cập nhật w** bằng gradient tính trên **toàn bộ** $n$ mẫu được gọi là **một lần lặp**. Cách làm này gọi là **Batch Gradient Descent**.
- $\mathbf{w}^{(0)}$ là giá trị khởi tạo, $\mathbf{w}^{(t)}$ là giá trị sau $t$ lần lặp, và ta mong $J\left(\mathbf{w}^{(t)}\right)$ giảm dần theo $t$: $\mathbf{w}^{(0)} \to \mathbf{w}^{(1)} \to \mathbf{w}^{(2)} \to \cdots \to \mathbf{w}^*$

### Thuật toán

**Đầu vào:** ma trận đặc trưng $\mathbf{X}$ (kích thước $n \times (d+1)$), vector giá trị thực $\mathbf{y}$ ($n \times 1$), tốc độ học $\eta > 0$, số lần lặp tối đa $T_{\max}$ và ngưỡng dừng $\varepsilon > 0$.

**Đầu ra:** vector tham số $\mathbf{w}$.

1. **Khởi tạo** $\mathbf{w} = \mathbf{0}$ (vector cột kích thước $(d+1) \times 1$) hoặc một giá trị nhỏ.
2. **Lặp** cho $t = 1,2, \ldots, T_{\max}$:
    1. Tính dự đoán $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ và sai số $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y}$.

2. Tính mất mát $J = \frac{1}{n}\mathbf{e}^T\mathbf{e}$.
3. Tính gradient $\nabla J = \frac{2}{n}\mathbf{X}^T\mathbf{e}$.
4. **Cập nhật** $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla J$.
5. **Nếu** điều kiện dừng thỏa mãn: **dừng**.

3. **Trả về** $\mathbf{w}$.

### Điều kiện dừng

Thuật toán dừng khi xảy ra một trong các điều kiện sau:

- **Mất mát gần như không giảm nữa:** $|J_{t+1} - J_t| < \varepsilon$.
- **Gradient gần bằng không:** $\| \nabla J \| < \varepsilon$. Khi đó ta đã ở gần điểm cực tiểu.
- **Đạt số lần lặp tối đa** $T_{\max}$. Đây là "van an toàn", cần thiết vì nếu $\eta$ chọn không phù hợp thì hai điều kiện đầu có thể không bao giờ xảy ra.

### Ba biến thể: Batch, SGD và Mini-batch

Công thức gradient $\nabla J = \frac{1}{n}\sum_i \nabla \ell_i$ cho phép ta tính gradient trên **một phần** của dữ liệu. Điều này dẫn đến ba biến thể có cùng cấu trúc:

| Biến thể | Số mẫu dùng cho mỗi lần cập nhật | Công thức cập nhật |
|---|---|---|
| **Batch GD** | Toàn bộ $n$ mẫu | $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\frac{2}{n}\mathbf{X}^T\mathbf{e}$ |
| **Stochastic GD (SGD)** | 1 mẫu | $\mathbf{w} \leftarrow \mathbf{w} - \eta\,2e_i\mathbf{x}_i^T$ |
| **Mini-batch GD** | $b$ mẫu ($1 < b < n$) | $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\frac{2}{b}\sum_{i\in\mathcal{B}} e_i\,\mathbf{x}_i^T$ |

Trong bảng, $\mathcal{B}$ là tập chỉ số của $b$ mẫu được chọn ngẫu nhiên. SGD và Mini-batch rẻ hơn mỗi lần cập nhật nhưng gradient "nhiễu" hơn, nên đường đi dao động. Chúng đặc biệt quan trọng khi $n$ rất lớn. Trong bài giảng này, ta dùng dạng **Batch** vì dễ tính tay hơn.

## 3.5. Ví dụ tính tay hai lần lặp

Ta huấn luyện trên 5 căn nhà với $\eta = 0{,}02$ và khởi tạo $\mathbf{w}^{(0)} = [0 \quad 0]^T$.

### Lần lặp 1

Với $\mathbf{w} = \mathbf{0}$, mô hình dự đoán $\hat{\mathbf{y}} = \mathbf{0}$ nên sai số $\mathbf{e} = -\mathbf{y}$:

$$\mathbf{e} = \begin{bmatrix} -1{,}2 \\ -1{,}9 \\ -2{,}4 \\ -3{,}2 \\ -3{,}3 \end{bmatrix}, \qquad \mathbf{X}^T\mathbf{e} = \begin{bmatrix} \sum_i e_i \\ \sum_i x_i\,e_i \end{bmatrix} = \begin{bmatrix} -12 \\ -65{,}5 \end{bmatrix}$$

Gradient và bộ tham số mới:

$$\nabla J = \frac{2}{5}\begin{bmatrix}-12\\-65{,}5\end{bmatrix} = \begin{bmatrix}-4{,}8\\-26{,}2\end{bmatrix}, \qquad \mathbf{w}^{(1)} = \begin{bmatrix}0\\0\end{bmatrix} - 0{,}02\begin{bmatrix}-4{,}8\\-26{,}2\end{bmatrix} = \begin{bmatrix}0{,}096\\0{,}524\end{bmatrix}$$

Mất mát giảm từ $J(\mathbf{w}^{(0)}) = 6{,}388$ xuống $J(\mathbf{w}^{(1)}) = 0{,}1242$.

### Lần lặp 2

Với $\mathbf{w}^{(1)}$, dự đoán và sai số là

$$\hat{\mathbf{y}} = \begin{bmatrix}1{,}668\\2{,}192\\2{,}716\\3{,}240\\3{,}764\end{bmatrix}, \qquad \mathbf{e} = \begin{bmatrix}0{,}468\\0{,}292\\0{,}316\\0{,}040\\0{,}464\end{bmatrix}, \qquad \mathbf{X}^T\mathbf{e} = \begin{bmatrix}1{,}58\\7{,}64\end{bmatrix}$$

$$\nabla J = \frac{2}{5}\begin{bmatrix}1{,}58\\7{,}64\end{bmatrix} = \begin{bmatrix}0{,}632\\3{,}056\end{bmatrix}, \qquad \mathbf{w}^{(2)} = \begin{bmatrix}0{,}096\\0{,}524\end{bmatrix} - 0{,}02\begin{bmatrix}0{,}632\\3{,}056\end{bmatrix} = \begin{bmatrix}0{,}08336\\0{,}46288\end{bmatrix}$$

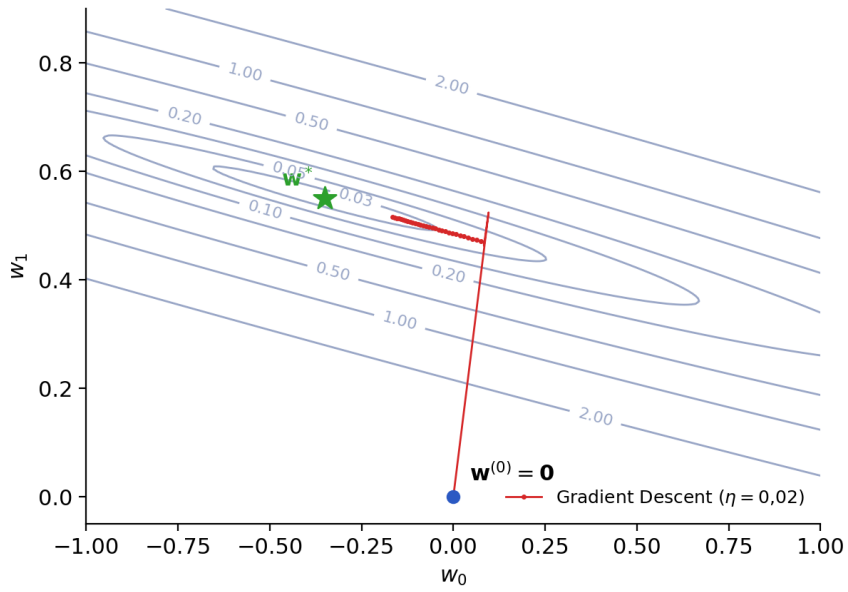
Mất mát tiếp tục giảm xuống $J(\mathbf{w}^{(2)}) = 0{,}0382$. Bảng sau cho mất mát qua các lần lặp (các giá trị sau lần 2 được tính bằng máy):

| Số lần lặp $t$ | 0 | 1 | 2 | 3 | 10 | 50 | 200 | 1000 |
|---|---|---|---|---|---|---|---|---|
| $J(\mathbf{w}^{(t)})$ | 6,388 | 0,1242 | 0,0382 | 0,0369 | 0,0364 | 0,0336 | 0,0275 | 0,0230 |

Sau 1000 lần lặp, $\mathbf{w}^{(1000)} \approx [-0{,}325 \quad 0{,}545]^T$, vẫn chưa hoàn toàn trùng với $\mathbf{w}^* = [-0{,}35 \quad 0{,}55]^T$ sẽ tìm được ở mục 3.7.

### Nhận xét: giảm nhanh rồi chậm lại

Bảng trên cho thấy $J$ **giảm rất nhanh** ở vài lần lặp đầu, rồi **chậm lại đáng kể**. Hình 4 giải thích hiện tượng này: các đường mức của $J$ là những hình elip rất **dài và hẹp**, tạo thành một "thung lũng". Bước đầu tiên đi rất nhanh xuống đáy thung lũng, nhưng sau đó thuật toán phải **bò dọc đáy thung lũng** với bước rất nhỏ.




*Hình 4. Các đường mức của $J(w_0, w_1)$ trên dữ liệu nhà và quỹ đạo của Gradient Descent với $\eta = 0{,}02$ (đường màu đỏ, vẽ 300 lần lặp). Ngôi sao xanh là $\mathbf{w}^*$.*

Nguyên nhân là đặc trưng $x$ có giá trị khá lớn (3 đến 7) so với cột toàn số 1. Hai tham số $w_0$ và $w_1$ vì vậy ảnh hưởng đến $J$ ở mức rất khác nhau: $J$ rất nhạy với $w_1$ nhưng ít nhạy với $w_0$. Tốc độ học buộc phải đủ nhỏ để ổn định theo hướng nhạy, và do đó đi chậm theo hướng kém nhạy.

## Cách khắc phục: chuẩn hóa đặc trưng

Ta **chuẩn hóa** đặc trưng để nó có trung bình 0 và độ lệch chuẩn 1:

$$z_i = \frac{x_i - \bar{x}}{\sigma}$$

Với dữ liệu nhà, $\bar{x} = 5$ và $\sigma = \sqrt{2}$ nên $z = [-1{,}414 \quad -0{,}707 \quad 0 \quad 0{,}707 \quad 1{,}414]$. Chạy Gradient Descent trên đặc trưng $z$ với $\eta = 0{,}3$ (lớn hơn 15 lần trước đây vì thung lũng đã "tròn" hơn), ta thu được

| Số lần lặp $t$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $J(\mathbf{w}^{(t)})$ | 6,388 | 1,0414 | 0,1859 | 0,0491 | 0,0272 | 0,0237 | 0,0231 |

Thuật toán hội tụ chỉ sau khoảng 17 lần lặp đến $\mathbf{w}' = [2{,}4 \quad 0{,}7778]^T$. Để quay về thang đo gốc, ta thay $z = (x - \bar{x})/\sigma$ vào $\hat{y} = w_0' + w_1' z$:

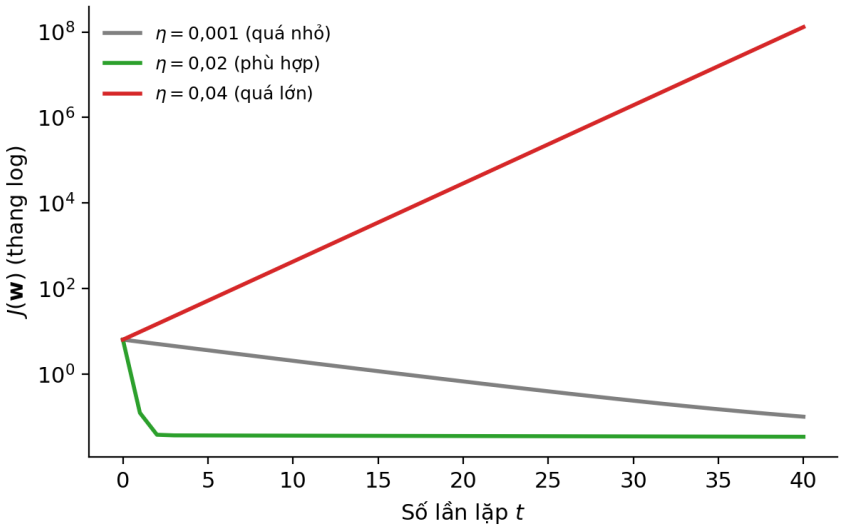
$$w_1 = \frac{w_1'}{\sigma} = \frac{0{,}7778}{\sqrt{2}} = 0{,}55, \qquad w_0 = w_0' - w_1' \frac{\bar{x}}{\sigma} = 2{,}4 - 0{,}7778 \cdot \frac{5}{\sqrt{2}} = -0{,}35$$

Ta thu lại đúng $\mathbf{w}^* = [-0{,}35 \quad 0{,}55]^T$. Lưu ý rằng $w_0' = 2{,}4$ chính là giá trung bình $\bar{y}$: khi đặc trưng đã được căn giữa, hệ số chệch bằng giá trị trung bình của đầu ra. Với nhiều đặc trưng,

**tham số của phép chuẩn hóa (trung bình, độ lệch chuẩn) phải được ước lượng trên tập huấn luyện** rồi áp dụng nhất quán cho dữ liệu kiểm tra và dữ liệu mới.

## 3.6. Vai trò của tốc độ học

Tốc độ học $\eta$ quyết định độ dài mỗi bước đi. Hình 5 vẽ $J$ theo số lần lặp trên dữ liệu nhà (đặc trưng chưa chuẩn hóa) với ba giá trị $\eta$.




*Hình 5. Ảnh hưởng của tốc độ học đến quá trình huấn luyện (trục tung dùng thang logarit).*

- $\eta$ **quá nhỏ** ($\eta = 0{,}001$): mỗi bước rất ngắn, $\mathbf{w}^{(t+1)} \approx \mathbf{w}^{(t)}$. Sau 40 lần lặp, $J$ vẫn còn $0{,}1009$, mô hình hội tụ **rất chậm**.
- $\eta$ **phù hợp** ($\eta = 0{,}02$): $J$ giảm mạnh ngay từ vài lần đầu và đi vào vùng gần cực tiểu.
- $\eta$ **quá lớn** ($\eta = 0{,}04$): mỗi bước vượt qua cực tiểu sang phía bên kia và **xa hơn** lần trước, nên $J$ **tăng** thay vì giảm: $6{,}388 \to 9{,}712 \to \cdots$. Sau 40 lần lặp, $J \approx 1{,}3 \times 10^8$. Thuật toán **phân kỳ**.

  **Ngưỡng ổn định (đọc thêm).** Với MSE, Gradient Descent hội tụ khi $0 < \eta < 2/L$, trong đó $L$ là giá trị riêng lớn nhất của ma trận $\frac{2}{n}\mathbf{X}^T\mathbf{X}$. Với dữ liệu nhà, $L \approx 55{,}86$ nên ngưỡng là $2/L \approx 0{,}0358$. Do đó $\eta = 0{,}02$ ổn định còn $\eta = 0{,}04 > 0{,}0358$ phân kỳ. Sau chuẩn hóa, ngưỡng này tăng lên 1 (với $\eta < 1$), đó là lý do ta dùng được $\eta = 0{,}3$.

Tóm lại, ta cần chọn $\eta$ sao cho vừa đủ lớn để học nhanh, vừa đủ nhỏ để không phân kỳ. Quan sát đường cong $J$ theo số lần lặp là cách đơn giản nhất để phát hiện $\eta$ chưa phù hợp.

## 3.7. Giải trực tiếp: phương trình chuẩn

### Có nhất thiết phải dùng Gradient Descent?

Không. Với hồi quy tuyến tính và MSE, ta có thể giải bài toán tối ưu **trực tiếp**. Tại điểm cực tiểu, gradient bằng $\mathbf{0}$:

$$\nabla J(\mathbf{w}) = \frac{2}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$$

Suy ra **phương trình chuẩn (normal equation)**:

$$\mathbf{X}^T\mathbf{X}\,\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

Nếu $\mathbf{X}^T\mathbf{X}$ khả nghịch (các cột của $\mathbf{X}$ độc lập tuyến tính), nghiệm duy nhất là

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Điều kiện khả nghịch không thỏa khi $n < d + 1$ (ít mẫu hơn tham số) hoặc khi có một đặc trưng là tổ hợp tuyến tính của các đặc trưng khác (ví dụ có cả diện tích tính bằng $m^2$ lẫn diện tích tính bằng $feet^2$). Khi đó phương trình chuẩn có vô số nghiệm hoặc cho kết quả không ổn định. Mục 5.2 sẽ cho thấy Ridge giải quyết vấn đề này.

## Ví dụ số với dữ liệu nhà

Với $\mathbf{X}$ và $\mathbf{y}$ ở mục 1.1:

$$\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 5 & 25 \\ 25 & 135 \end{bmatrix}, \qquad \mathbf{X}^T\mathbf{y} = \begin{bmatrix} 12 \\ 65{,}5 \end{bmatrix}$$

Định thức của $\mathbf{X}^T\mathbf{X}$ là $5 \cdot 135 - 25 \cdot 25 = 50 \neq 0$ nên

$$\mathbf{w}^* = \frac{1}{50}\begin{bmatrix} 135 & -25 \\ -25 & 5 \end{bmatrix}\begin{bmatrix} 12 \\ 65{,}5 \end{bmatrix} = \frac{1}{50}\begin{bmatrix} 1620 - 1637{,}5 \\ -300 + 327{,}5 \end{bmatrix} = \begin{bmatrix} -0{,}35 \\ 0{,}55 \end{bmatrix}$$

Đây chính là đường A, đường có $J = 0{,}023$ nhỏ nhất. Gradient Descent ở mục 3.5 sau khi chuẩn hóa cũng dẫn tới đúng nghiệm này.

## Ý nghĩa hình học của nghiệm tối ưu

Từ $\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) = \mathbf{0}$ ta có $\mathbf{X}^T\mathbf{e} = \mathbf{0}$, tức là vector sai số **vuông góc với mọi cột của X**. Vì cột đầu của $\mathbf{X}$ toàn số 1, điều này có hai hệ quả cụ thể:

- $\sum_i e_i = 0$: tổng sai số bằng 0, mô hình không dự đoán lệch về một phía.
- $\sum_i x_i\, e_i = 0$: sai số không còn tương quan với đặc trưng.

Kiểm tra với đường A: $\sum_i e_i = 0{,}10 - 0{,}05 + 0 - 0{,}25 + 0{,}20 = 0$ và $\sum_i x_i\, e_i = 3 \cdot 0{,}10 + 4 \cdot (-0{,}05) + 0 + 6 \cdot (-0{,}25) + 7 \cdot 0{,}20 = 0{,}3 - 0{,}2 - 1{,}5 + 1{,}4 = 0$. Đúng như vậy.

## Gradient Descent hay phương trình chuẩn?

| Tiêu chí | Gradient Descent | Phương trình chuẩn |
|---|---|---|
| Cách tìm $\mathbf{w}^*$ | Lặp từng bước, tiến dần đến nghiệm | Giải một hệ phương trình, được nghiệm chính xác |
| Siêu tham số | Cần chọn $\eta$, $T_{\max}$, $\varepsilon$ | Không cần |
| Chi phí khi số đặc trưng $d$ lớn | Mỗi lần lặp tốn khoảng $n \cdot d$ phép tính | Tốn khoảng $nd^2 + d^3$ phép tính |

| Tiêu chí | Gradient Descent | Phương trình chuẩn |
|---|---|---|
| Áp dụng cho hàm mất mát khác | Có (MAE, Lasso, mạng nơ-ron, …) | Chỉ MSE (và Ridge) |
| Chuẩn hóa đặc trưng | Nên có, để hội tụ nhanh | Không bắt buộc |

### Mô hình, hàm mất mát và thuật toán là ba thứ khác nhau

Từ các phân tích trên, ta rút ra một nhận xét quan trọng:

> **Hồi quy tuyến tính không phải là Gradient Descent.** Hồi quy tuyến tính là **mô hình**. MSE là **hàm mất mát**. Gradient Descent và phương trình chuẩn là **hai cách khác nhau để tối ưu** hàm mất mát đó.

Việc phân biệt rõ ba thành phần này giúp ta thay đổi từng thành phần mà không phải bắt đầu lại từ đầu, như sẽ thấy ở các mục 4 và 5.

## Tóm tắt mục 3

- Huấn luyện là bài toán **tìm** $\mathbf{w}^* = \text{argmin}_{\mathbf{w}} J(\mathbf{w})$, với $J$ là hàm mất mát trên dữ liệu huấn luyện.
- **MSE**: $J(\mathbf{w}) = \frac{1}{n}\sum_i \ell_i$ với $\ell_i = e_i^2$, viết gọn là $J = \frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|^2$. $J$ là hàm bậc hai, hình cái bát, có một cực tiểu duy nhất.
- **Gradient**: $\nabla J = \frac{2}{n}\mathbf{X}^T(\mathbf{Xw} - \mathbf{y})$, và $-\nabla J$ là hướng giảm nhanh nhất. **Gradient Descent**: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$.
- Tốc độ học $\eta$ quá nhỏ thì hội tụ chậm, quá lớn thì phân kỳ. **Chuẩn hóa đặc trưng** giúp Gradient Descent hội tụ nhanh và cho phép dùng $\eta$ lớn hơn.
- **Phương trình chuẩn** $\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ cho nghiệm trực tiếp khi $\mathbf{X}^T\mathbf{X}$ khả nghịch. Tại nghiệm, $\mathbf{X}^T\mathbf{e} = \mathbf{0}$.
- **Mô hình** + **Hàm mất mát** + **Thuật toán tối ưu** = **Huấn luyện**, và ba thành phần này độc lập về mặt khái niệm.

## Câu hỏi kiểm tra nhanh

1. Tính MSE của đường B ($\mathbf{w} = [0{,}2 \quad 0{,}35]^T$) trên 5 căn nhà, biết các giá trị dự đoán $\hat{\mathbf{y}} = [1{,}25 \quad 1{,}60 \quad 1{,}95 \quad 2{,}30 \quad 2{,}65]^T$.
2. Cho ba mẫu có $\mathbf{x}_1 = [1 \quad 2]$, $\mathbf{x}_2 = [1 \quad 4]$, $\mathbf{x}_3 = [1 \quad 6]$ và $\mathbf{y} = [2{,}5 \quad 3 \quad 3{,}5]^T$. Với $\mathbf{w} = [1 \quad 0{,}5]^T$, hãy tính $J$, $\nabla J$ và $\mathbf{w}$ sau một lần cập nhật với $\eta = 0{,}1$.
3. Hãy dùng phương trình chuẩn để tìm $\mathbf{w}^*$ cho ba mẫu ở câu 2. Giá trị $J(\mathbf{w}^*)$ là bao nhiêu?
4. Vì sao $\eta = 0{,}04$ làm Gradient Descent phân kỳ trên dữ liệu nhà còn $\eta = 0{,}02$ thì không?
5. Vì sao nói "hồi quy tuyến tính không phải là Gradient Descent"? Chuẩn hóa đặc trưng giúp ích gì cho Gradient Descent?

### Gợi ý đáp án

1. $\mathbf{e} = \hat{\mathbf{y}} - \mathbf{y} = [0{,}05 \quad -0{,}30 \quad -0{,}45 \quad -0{,}90 \quad -0{,}65]^T$. Tổng bình phương: $0{,}0025 + 0{,}09 + 0{,}2025 + 0{,}81 + 0{,}4225 = 1{,}5275$. Vậy $J = 1{,}5275/5 = 0{,}3055$.

2. $\hat{\mathbf{y}} = [2 \quad 3 \quad 4]^T$ nên $\mathbf{e} = [-0{,}5 \quad 0 \quad 0{,}5]^T$ và $J = \frac{0{,}25+0+0{,}25}{3} = \frac{1}{6} \approx 0{,}1667$. Ta có $\mathbf{X}^T\mathbf{e} = \begin{bmatrix} \sum e_i \\ \sum x_i e_i \end{bmatrix} = \begin{bmatrix} 0 \\ -1+0+3 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, nên $\nabla J = \frac{2}{3}\begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 4/3 \end{bmatrix}$. Cập nhật: $\mathbf{w} \leftarrow \begin{bmatrix} 1 \\ 0{,}5 \end{bmatrix} - 0{,}1\begin{bmatrix} 0 \\ 4/3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0{,}3667 \end{bmatrix}$.
3. $\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 3 & 12 \\ 12 & 56 \end{bmatrix}$ (định thức 24) và $\mathbf{X}^T\mathbf{y} = \begin{bmatrix} 9 \\ 38 \end{bmatrix}$. Suy ra $\mathbf{w}^* = \frac{1}{24}\begin{bmatrix} 56 & -12 \\ -12 & 3 \end{bmatrix}\begin{bmatrix} 9 \\ 38 \end{bmatrix} = \frac{1}{24}\begin{bmatrix} 48 \\ 6 \end{bmatrix} = \begin{bmatrix} 2 \\ 0{,}25 \end{bmatrix}$. Ba điểm nằm đúng trên đường thẳng $\hat{y} = 2 + 0{,}25x$ nên $J(\mathbf{w}^*) = 0$.
4. Gradient Descent trên MSE hội tụ khi $\eta < 2/L$, với $L$ là giá trị riêng lớn nhất của $\frac{2}{n}\mathbf{X}^T\mathbf{X}$. Với dữ liệu nhà, $L \approx 55{,}86$ nên ngưỡng $\approx 0{,}0358$. Vì $0{,}02 < 0{,}0358 < 0{,}04$ nên chỉ $\eta = 0{,}02$ hội tụ.
5. Hồi quy tuyến tính là **mô hình** $\hat{y} = \mathbf{xw}$, còn Gradient Descent chỉ là một **cách tối ưu** để tìm $\mathbf{w}$. Ta có thể tìm cùng mô hình đó bằng phương trình chuẩn. Chuẩn hóa đặc trưng làm các đường mức của $J$ bớt dài và hẹp, nên Gradient Descent hội tụ nhanh hơn và cho phép dùng $\eta$ lớn hơn.

# 4. Mở rộng mô hình bằng biến đổi đặc trưng

Ở mục 3, ta đã có một quy trình huấn luyện hoàn chỉnh cho mô hình $\hat{y} = \mathbf{xw}$. Mô hình này đơn giản, nhưng nó có một giới hạn: giá trị dự đoán chỉ là **tổ hợp tuyến tính** của các đặc trưng ban đầu, nên chỉ mô tả được quan hệ có dạng đường thẳng (hoặc mặt phẳng). Trong thực tế nhiều quan hệ là **cong**. Mục này trả lời ba câu hỏi:

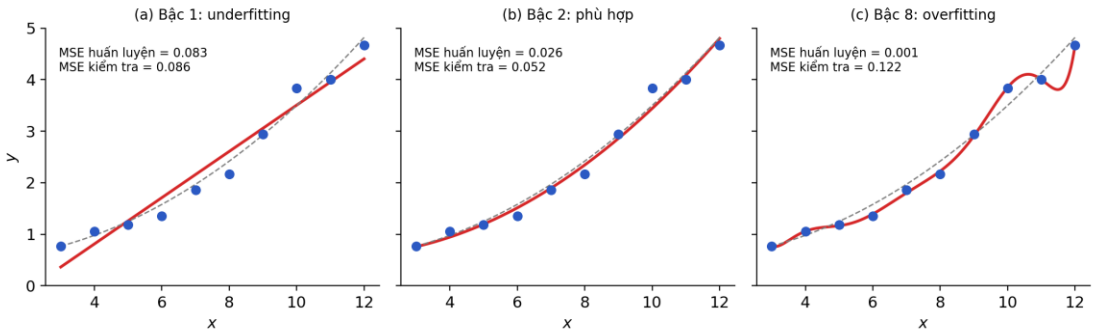
- Khi quan hệ giữa $x$ và $y$ không phải đường thẳng, mô hình tuyến tính còn dùng được không?
- Biến đổi đặc trưng là gì, và các công thức huấn luyện ở mục 3 thay đổi như thế nào?
- Tăng độ phức tạp của mô hình có luôn làm mô hình tốt hơn không?

Ta tiếp tục dùng quy ước ký hiệu ở mục 1: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là vector hàng, $\mathbf{w}$ là vector cột.

## 4.1. Khi quan hệ không phải là đường thẳng

Xét bài toán dự đoán giá nhà theo diện tích, nhưng lần này với các căn nhà có diện tích từ 30 đến 120 $m^2$ ($x$ từ 3 đến 12 theo đơn vị 10 $m^2$). Trong thực tế, giá nhà thường tăng **nhanh hơn** khi diện tích lớn: nhà lớn có giá mỗi $m^2$ cao hơn. Quan hệ này có dạng cong, và Hình 6a cho thấy một đường thẳng bám dữ liệu rất kém ở hai đầu: nó dự đoán thấp ở giữa và cao ở hai đầu, hoặc ngược lại.

Hiện tượng này gọi là **underfitting (dưới khớp)**: mô hình **quá đơn giản** để nắm bắt cấu trúc của dữ liệu. Câu hỏi là ta có thể cải thiện mà **không phải thay đổi cơ chế huấn luyện** hay không. Câu trả lời là có, nhờ biến đổi đặc trưng.




*Hình 6. Ba mô hình đa thức trên cùng một dữ liệu cong (*10 *điểm huấn luyện màu xanh, đường nét đứt xám là quan hệ thật): (a) bậc* 1 *(dưới khớp), (b) bậc* 2 *(phù hợp), (c) bậc* 8 *(quá khớp). Số liệu ghi trong từng hình là MSE trên tập huấn luyện và trên* 200 *điểm kiểm tra mới.*

## 4.2. Ý tưởng biến đổi đặc trưng

### Từ không gian đầu vào sang không gian đặc trưng

Thay vì đưa thẳng $x$ vào mô hình, ta **tạo thêm các đặc trưng mới** là hàm của $x$, rồi vẫn dùng mô hình tuyến tính trên các đặc trưng mới. Gọi $\boldsymbol{\phi}$ là một phép biến đổi biến vector đặc trưng $\mathbf{x}$ thành vector hàng gồm $m$ đặc trưng mới, có phần tử đầu tiên bằng 1 (để giữ hệ số chệch):

$$\boldsymbol{\phi}(\mathbf{x}) = [1 \quad \phi_1(\mathbf{x}) \quad \phi_2(\mathbf{x}) \quad \cdots \quad \phi_m(\mathbf{x})]$$

Mỗi $\phi_j$ là một hàm của đặc trưng đầu vào, có thể là hàm đa thức, hàm lượng giác, hàm mũ, hàm logarit hoặc một hàm thiết kế theo kiến thức chuyên ngành. Khi đó mô hình trở thành

$$\hat{y} = \boldsymbol{\phi}(\mathbf{x})\mathbf{w} = w_0 + \sum_{j=1}^{m} w_j\, \phi_j(\mathbf{x}), \qquad \mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_m]^T$$

Điểm cần nhấn mạnh: các hàm $\phi_j$ có thể **phi tuyến theo x**, nhưng mô hình vẫn **tuyến tính theo các tham số w**. Đây chính là lý do ta vẫn gọi đó là hồi quy tuyến tính và vẫn dùng được toàn bộ công cụ ở mục 3.

### Ví dụ với một đặc trưng: đa thức

Với một đặc trưng $x$, ta chọn $\boldsymbol{\phi}(x) = [1 \quad x \quad x^2 \quad x^3]$. Mô hình là

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

Dữ liệu ban đầu có 1 chiều, nhưng sau biến đổi có 3 đặc trưng (chưa tính số 1). Phép biến đổi **không tạo ra thông tin mới**. Nó chỉ cho mô hình một cách biểu diễn khác, trong đó đường cong trở thành một "mặt phẳng" trong không gian đặc trưng mới.

### Ví dụ với hai đặc trưng: thêm bậc hai và tương tác

Với hai đặc trưng $x_1, x_2$, ta chọn

$$\boldsymbol{\phi}(x_1, x_2) = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1 x_2 \quad x_2^2]$$

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2$$

So với mô hình ban đầu, mô hình mới biểu diễn được quan hệ cong theo $x_1$ (nhờ $x_1^2$), cong theo $x_2$ (nhờ $x_2^2$) và **tác động kết hợp** giữa hai đặc trưng (nhờ $x_1 x_2$). Chẳng hạn, nếu $x_1$ là diện tích và $x_2$ là khoảng cách đến trung tâm, tác động của diện tích lên giá nhà có thể khác nhau tùy vị trí, và thành phần $x_1 x_2$ giúp mô hình biểu diễn một phần sự tương tác đó.

## Các hàm biến đổi thường gặp

| Nhóm đặc trưng | Ví dụ | Khả năng biểu diễn |
|---|---|---|
| Đa thức | $x,\ x^2,\ x^3$ | Quan hệ cong |
| Tương tác | $x_1 x_2$ | Tác động kết hợp giữa các đặc trưng |
| Lượng giác | $\sin x,\ \cos x$ | Quan hệ tuần hoàn |
| Logarit | $\log(1+x)$ | Quan hệ tăng chậm, bão hòa |
| Hàm mũ | $e^x$ | Quan hệ tăng hoặc giảm theo hàm mũ |
| Hàm cơ sở xuyên tâm | $e^{-\gamma(x-c)^2}$ | Quan hệ cục bộ quanh tâm $c$ |

Không phải bài toán nào cũng cần mọi nhóm. Mục tiêu là chọn cách biểu diễn giúp mô hình học được cấu trúc quan trọng của dữ liệu mà không làm nó quá phức tạp.

## Dạng ma trận sau biến đổi

Với $n$ mẫu, ta thay ma trận $\mathbf{X}$ bằng **ma trận đặc trưng mới $\boldsymbol{\Phi}$** có kích thước $n \times (m+1)$, mà hàng thứ $i$ là $\boldsymbol{\phi}(\mathbf{x}_i)$:

$$\boldsymbol{\Phi} = \begin{bmatrix} 1 & \phi_1(\mathbf{x}_1) & \cdots & \phi_m(\mathbf{x}_1) \\ 1 & \phi_1(\mathbf{x}_2) & \cdots & \phi_m(\mathbf{x}_2) \\ \vdots & \vdots & & \vdots \\ 1 & \phi_1(\mathbf{x}_n) & \cdots & \phi_m(\mathbf{x}_n) \end{bmatrix} = \begin{bmatrix} \boldsymbol{\phi}(\mathbf{x}_1) \\ \boldsymbol{\phi}(\mathbf{x}_2) \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}_n) \end{bmatrix}$$

**Ví dụ.** Với 5 căn nhà ở mục 1.1 và $\boldsymbol{\phi}(x) = [1 \quad x \quad x^2]$:

$$\boldsymbol{\Phi} = \begin{bmatrix} 1 & 3 & 9 \\ 1 & 4 & 16 \\ 1 & 5 & 25 \\ 1 & 6 & 36 \\ 1 & 7 & 49 \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

Khi đó mọi công thức ở mục 3 giữ nguyên, chỉ cần **thay X bằng $\boldsymbol{\Phi}$**:

| Đại lượng | Công thức |
|---|---|
| Dự đoán | $\hat{\mathbf{y}} = \boldsymbol{\Phi}\mathbf{w}$ |
| Sai số | $\mathbf{e} = \boldsymbol{\Phi}\mathbf{w} - \mathbf{y}$ |
| Hàm mất mát MSE | $J(\mathbf{w}) = \frac{1}{n} \parallel \boldsymbol{\Phi}\mathbf{w} - \mathbf{y} \parallel^2$ |

| Đại lượng | Công thức |
|---|---|
| Gradient | $\nabla J(\mathbf{w}) = \frac{2}{n}\mathbf{\Phi}^T(\mathbf{\Phi w} - \mathbf{y})$ |
| Cập nhật | $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\frac{2}{n}\mathbf{\Phi}^T(\mathbf{\Phi w} - \mathbf{y})$ |
| Phương trình chuẩn | $\mathbf{w}^* = (\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{y}$ |

Ta chỉ thay đổi **cách biểu diễn dữ liệu**, còn cơ chế huấn luyện không đổi. Trong ma trận $\mathbf{\Phi}$ ở ví dụ trên, các cột có thang đo rất khác nhau (1, 3 đến 7, 9 đến 49), nên việc **chuẩn hóa** các cột (trừ cột 1) càng quan trọng, đặc biệt khi dùng Gradient Descent.

## 4.3. Quá khớp: khi mô hình quá phức tạp

### Ba mô hình trên cùng một dữ liệu

Ta quay lại dữ liệu cong ở mục 4.1 với 10 điểm huấn luyện (có nhiễu với độ lệch chuẩn 0,25), và so sánh ba mô hình đa thức bậc 1, 2 và 8. Các đặc trưng đa thức được chuẩn hóa trước khi huấn luyện. Ngoài 10 điểm huấn luyện, ta dùng thêm 200 điểm **chưa từng thấy** để kiểm tra.

| Mô hình | Số tham số | MSE huấn luyện | MSE kiểm tra | $\| \mathbf{w}_{1:m} \|$ (độ lớn các trọng số) |
|---|---|---|---|---|
| Bậc 1 | 2 | 0,0829 | 0,0861 | 1,29 |
| Bậc 2 | 3 | 0,0256 | **0,0521** | 1,45 |
| Bậc 8 | 9 | **0,0009** | 0,1221 | 182 752 |

Từ bảng và Hình 6 ta rút ra:

- **Bậc 1 (underfitting):** sai số lớn trên **cả** tập huấn luyện và tập kiểm tra. Mô hình quá đơn giản.
- **Bậc 2 (phù hợp):** sai số kiểm tra thấp nhất, gần mức nhiễu của dữ liệu ($0,25^2 = 0,0625$). Mô hình có cùng dạng với quan hệ thật.
- **Bậc 8 (overfitting):** sai số huấn luyện gần bằng 0 nhưng sai số kiểm tra **tăng vọt**. Đường cong uốn lượn để đi qua từng điểm, tức là **học cả nhiễu** của dữ liệu huấn luyện. Dấu hiệu nhận biết là các trọng số có độ lớn khổng lồ (hàng trăm nghìn), triệt tiêu lẫn nhau để tạo ra những dao động mạnh.

Hiện tượng bậc 8 gọi là **overfitting (quá khớp)**: mô hình **quá phức tạp** so với lượng dữ liệu, ghi nhớ nhiễu thay vì học quy luật.

### Vì sao phải đo trên dữ liệu chưa thấy?

Nếu chỉ nhìn sai số huấn luyện, ta sẽ luôn chọn mô hình phức tạp nhất vì thêm đặc trưng chỉ có thể làm $J$ trên dữ liệu huấn luyện giảm hoặc giữ nguyên. Điều ta thực sự quan tâm là **khả năng tổng quát hóa**: mô hình dự đoán tốt trên dữ liệu **mới**. Vì vậy ta luôn dành ra một **tập kiểm tra (test set)** không dùng khi huấn luyện, và đánh giá mô hình trên đó (mục 6.2).

### Giới hạn của việc tăng số đặc trưng

- **Số đặc trưng bùng nổ.** Với $d$ đặc trưng gốc và đa thức bậc $p$, số đặc trưng mới (không tính hằng số) là $\binom{d+p}{p} - 1$. Với $d = 10$ và $p = 3$ ta có 285 đặc trưng, với $d = 100$ và $p = 3$ ta có 176 850 đặc trưng.
- **Đặc trưng tương quan mạnh.** Các cột như $x$, $x^2$, $x^3$ thường tương quan mạnh, khiến $\boldsymbol{\Phi}^T\boldsymbol{\Phi}$ gần suy biến và nghiệm không ổn định.
- **Dễ quá khớp** khi số đặc trưng $m$ gần hoặc vượt số mẫu $n$.

Cách kiểm soát gồm: chuẩn hóa đặc trưng, chọn bậc đa thức bằng dữ liệu kiểm chứng, và quan trọng nhất là **điều chuẩn (regularization)**, nội dung của mục 5.2.

> **Kết luận.** Biến đổi đặc trưng là cầu nối từ hồi quy tuyến tính cơ bản sang các mô hình biểu diễn được quan hệ phi tuyến. Ta mở rộng cách biểu diễn dữ liệu nhưng giữ nguyên khung huấn luyện: dự đoán, tính mất mát, tính gradient, cập nhật tham số và đánh giá khả năng tổng quát hóa. Cần phân biệt rõ: mô hình **phi tuyến theo đặc trưng** nhưng **tuyến tính theo tham số**.

## Tóm tắt mục 4

- Hồi quy tuyến tính trên đặc trưng gốc chỉ biểu diễn được quan hệ dạng đường thẳng (mặt phẳng). Khi quan hệ cong, mô hình bị **underfitting**.
- **Biến đổi đặc trưng $\mathbf{x} \mapsto \boldsymbol{\phi}(\mathbf{x})$** (đa thức, tương tác, lượng giác, logarit, …) cho phép biểu diễn quan hệ phi tuyến, mà mô hình vẫn **tuyến tính theo tham số $\mathbf{w}$**.
- Sau biến đổi, ta chỉ cần **thay $\mathbf{X}$ bằng $\boldsymbol{\Phi}$** trong mọi công thức ở mục 3: $\hat{\mathbf{y}} = \boldsymbol{\Phi}\mathbf{w}$, $\nabla J = \frac{2}{n}\boldsymbol{\Phi}^T(\boldsymbol{\Phi}\mathbf{w} - \mathbf{y})$.
- Mô hình quá phức tạp bị **overfitting**: sai số huấn luyện rất nhỏ nhưng sai số kiểm tra lớn, và các trọng số rất lớn. Phải đánh giá trên **dữ liệu chưa thấy**.
- Số đặc trưng đa thức tăng rất nhanh theo $d$ và $p$, nên cần chuẩn hóa, chọn bậc bằng dữ liệu kiểm chứng và dùng điều chuẩn.

## Câu hỏi kiểm tra nhanh

1. Với $\boldsymbol{\phi}(x) = [1 \quad x \quad x^2]$ và $\mathbf{w} = [1 \quad 0{,}5 \quad 0{,}1]^T$, hãy tính $\hat{y}$ tại $x = 4$.
2. Với $\boldsymbol{\phi}(x_1, x_2) = [1 \quad x_1 \quad x_2 \quad x_1x_2]$ và $\mathbf{w} = [1 \quad 2 \quad 3 \quad -1]^T$, hãy tính $\hat{y}$ tại $\mathbf{x} = [2 \quad 3]$.
3. Mô hình $\hat{y} = w_0 + w_1 \sin x$ có phải là mô hình hồi quy tuyến tính không? Vì sao?
4. Viết ma trận $\boldsymbol{\Phi}$ cho ba mẫu $x = 1{,}2{,}3$ với $\boldsymbol{\phi}(x) = [1 \quad x \quad x^2]$. Vì sao trường hợp này (số mẫu bằng số tham số) là một cảnh báo về quá khớp?
5. Mô hình bậc 8 có MSE huấn luyện $0{,}0009$ và MSE kiểm tra $0{,}1221$. Mô hình bậc 2 có $0{,}0256$ và $0{,}0521$. Mô hình nào tốt hơn? Hiện tượng của mô hình bậc 8 là gì?
6. Có bao nhiêu đặc trưng đa thức (không tính hằng số) khi $d = 10$ đặc trưng gốc và bậc $p = 3$?

### Gợi ý đáp án

1. $\hat{y} = 1 + 0{,}5 \cdot 4 + 0{,}1 \cdot 16 = 1 + 2 + 1{,}6 = 4{,}6$.

2. $\hat{y} = 1 + 2 \cdot 2 + 3 \cdot 3 - 1 \cdot (2 \cdot 3) = 1 + 4 + 9 - 6 = 8.$
3. **Có.** Đặt $\phi_1(x) = \sin x$ thì $\hat{y} = w_0 + w_1\phi_1(x)$ tuyến tính theo các tham số $w_0, w_1$, dù phi tuyến theo $x$.
4. $\mathbf{\Phi} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix}$. Ma trận này vuông và khả nghịch, nên với **bất kỳ y** nào cũng tồn tại **w** cho $J = 0$. Mô hình khớp hoàn hảo cả nhiễu, nên $J = 0$ không nói lên chất lượng thật.
5. Mô hình **bậc 2 tốt hơn**, vì ta đánh giá bằng sai số trên dữ liệu chưa thấy ($0{,}0521 < 0{,}1221$). Mô hình bậc 8 bị **overfitting**: sai số huấn luyện rất nhỏ nhưng sai số kiểm tra lớn.
6. $\binom{13}{3} - 1 = 286 - 1 = 285$ đặc trưng.

# 5. Mở rộng hàm mất mát và điều chuẩn

Ở mục 3, ta cố định hàm mất mát là MSE. Nhưng việc chọn hàm mất mát không đơn thuần là chọn một công thức. Nó phản ánh **ta muốn mô hình ưu tiên điều gì**. Mục này giải quyết hai vấn đề đã gặp ở các mục trước:

- Dữ liệu có **ngoại lệ (outlier)** làm lệch mô hình, vì MSE phạt rất mạnh sai số lớn.
- Mô hình quá phức tạp bị **quá khớp** với các trọng số rất lớn (mục 4.3).

Ta trả lời ba câu hỏi:

- Làm thế nào để mô hình bớt nhạy với ngoại lệ? (MAE)
- Làm thế nào để kiểm soát độ phức tạp của mô hình? (Ridge và Lasso)
- Hàm mục tiêu, gradient và cách huấn luyện của từng trường hợp là gì?

Các công thức dưới đây viết với ma trận **X**. Nếu đã biến đổi đặc trưng ở mục 4, ta chỉ cần thay **X** bằng **Φ**. Ta dùng chung $\mathbf{e} = \mathbf{Xw} - \mathbf{y}$ làm điểm xuất phát.

## 5.1. Từ MSE đến MAE

### MSE phạt sai số lớn rất mạnh

Xét hai sai số $e_1 = 2$ và $e_2 = 10$. Sai số thứ hai lớn gấp 5 lần, nhưng sau khi bình phương, mức phạt lớn gấp $\frac{100}{4} = 25$ lần. Điều này vừa là **ưu điểm** (mô hình cố gắng tránh sai số lớn), vừa là **hạn chế**: nếu một quan sát có sai số rất lớn vì nó là ngoại lệ hoặc dữ liệu bị nhiễu, quan sát đó ảnh hưởng quá mạnh đến quá trình huấn luyện.

### MAE: dùng giá trị tuyệt đối thay vì bình phương

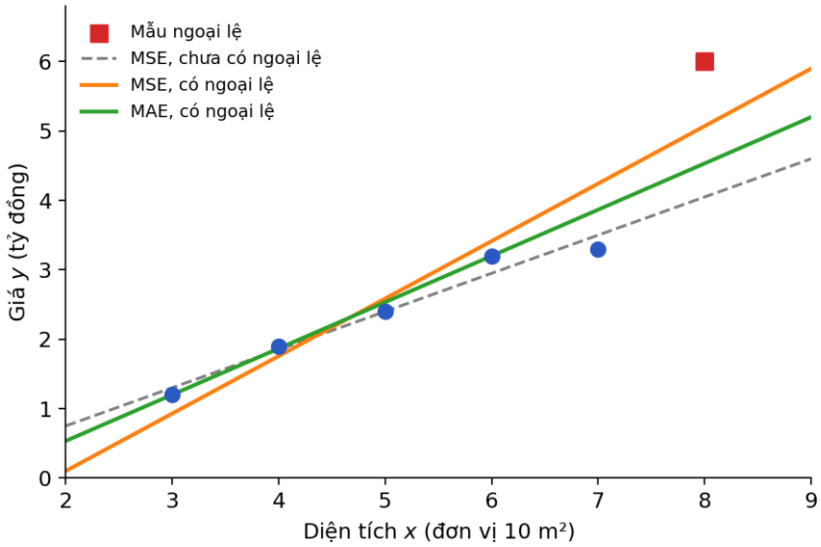
**MAE (Mean Absolute Error, sai số tuyệt đối trung bình)** dùng giá trị tuyệt đối của sai số:

$$J_{\text{MAE}}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}|e_i| = \frac{1}{n}\,\|\,\mathbf{Xw} - \mathbf{y}\,\|_1, \qquad \|\,\mathbf{e}\,\|_1 = \sum_{i=1}^{n}|e_i|$$

Với sai số 2 và 10, mức phạt lần lượt là 2 và 10, tức là tăng **tuyến tính** theo độ lớn sai số thay vì tăng vọt.

## Ví dụ: một căn nhà ngoại lệ

Thêm vào 5 căn nhà của ví dụ xuyên suốt một căn thứ sáu có $x = 8$ ($80\ m^2$) và giá $y = 6{,}0$ tỷ đồng, một giá bất thường (có thể do nhập sai số liệu hoặc đây là nhà có vị trí đặc biệt). Ta huấn luyện lại bằng MSE và bằng MAE.




*Hình 7. Ảnh hưởng của một mẫu ngoại lệ (ô vuông đỏ) lên đường hồi quy. Đường MSE (cam) bị kéo lệch nhiều hơn đường MAE (xanh lá). Đường nét đứt xám là đường MSE khi chưa có ngoại lệ.*

| Cách huấn luyện | $w_0$ | $w_1$ | MAE trên 5 căn nhà bình thường |
|---|---|---|---|
| MSE, chưa có ngoại lệ | $-0{,}35$ | 0,55 | 0,1200 |
| MSE, có ngoại lệ | $-1{,}557$ | 0,829 | 0,3515 |
| MAE, có ngoại lệ | $-0{,}80$ | 0,667 | 0,1467 |

Với MSE, một mẫu ngoại lệ làm hệ số góc tăng từ 0,55 lên 0,829 (tăng 51%), và sai số trung bình trên 5 căn nhà bình thường **tăng gần gấp ba** (0,12 → 0,3515). Với MAE, đường hồi quy vẫn bám sát nhóm nhà bình thường (sai số 0,1467). Lý do sẽ rõ khi so sánh gradient.

## Gradient của MAE

Hàm $|e|$ **không khả vi** tại $e = 0$. Với $e \neq 0$, đạo hàm bằng hàm dấu: $\frac{d|e|}{de} = \text{sign}(e)$. Tại $e = 0$, ta dùng **dưới đạo hàm (subgradient)** và chọn $\text{sign}(0) = 0$. Đặt $\text{sign}(\mathbf{e})$ là vector áp dụng hàm dấu lên từng phần tử của $\mathbf{e}$, một dưới đạo hàm của MAE là

$$\partial J_{\text{MAE}}(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T \text{sign}(\mathbf{X}\mathbf{w} - \mathbf{y})$$

Đặt cạnh gradient của MSE, ta thấy khác biệt then chốt:

$$\nabla J_{\text{MSE}}(\mathbf{w}) = \frac{2}{n}\mathbf{X}^T\mathbf{e}, \qquad \partial J_{\text{MAE}}(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T\text{sign}(\mathbf{e})$$

**MSE dùng độ lớn của sai số, còn MAE chỉ dùng dấu của sai số.** Một mẫu có sai số $e_i = 10$ đóng góp $2 \cdot 10 = 20$ vào gradient của MSE nhưng chỉ đóng góp 1 vào gradient của MAE. Đó là lý do ngoại lệ chi phối MSE nhưng không chi phối MAE.

Quy tắc cập nhật với MAE dùng dưới đạo hàm: $\mathbf{w} \leftarrow \mathbf{w} - \eta\ \partial J_{\text{MAE}}$. Vì độ lớn bước không giảm khi tiến gần cực tiểu, nên thường cần giảm dần $\eta$ theo thời gian.

### So sánh MSE và MAE

| Đặc điểm | MSE | MAE |
|---|---|---|
| Sai số được sử dụng | $e^2$ | $\lvert e \rvert$ |
| Ảnh hưởng của sai số lớn | Rất mạnh | Nhẹ hơn (tăng tuyến tính) |
| Độ nhạy với ngoại lệ | Cao | Thấp |
| Gradient | $\frac{2}{n}\mathbf{X}^T\mathbf{e}$, dùng **độ lớn** sai số | $\frac{1}{n}\mathbf{X}^T\text{sign}(\mathbf{e})$, dùng **dấu** sai số |
| Độ trơn | Khả vi mọi nơi | Không khả vi tại $e = 0$ |
| Nghiệm đóng | Có (phương trình chuẩn) | Không có công thức đơn giản |

Từ đây ta rút ra: **thay đổi hàm mất mát là thay đổi tiêu chí đánh giá mô hình**, và có thể kéo theo thay đổi cả thuật toán tối ưu.

## 5.2. Điều chuẩn: kiểm soát độ phức tạp của mô hình

### Ý tưởng: thêm một "chi phí" cho sự phức tạp

MAE vẫn chỉ đo sai số giữa dự đoán và giá trị thực. Ta có thể thêm vào hàm mục tiêu một thành phần để **yêu cầu mô hình thỏa mãn một đặc tính mong muốn**. Ở đây đặc tính đó là các trọng số không quá lớn. Đây là ý tưởng của **điều chuẩn (regularization)**:

$$J_{\text{reg}}(\mathbf{w}) = J_{\text{MSE}}(\mathbf{w}) + \lambda\, R(\mathbf{w})$$

Trong đó:

- $J_{\text{MSE}}(\mathbf{w})$ đo mức sai lệch dự đoán trên dữ liệu;
- $R(\mathbf{w})$ là **thành phần điều chuẩn (penalty)**, đo độ lớn của các trọng số;
- $\lambda \geq 0$ là tham số điều khiển mức độ điều chuẩn.

Khi đó quá trình huấn luyện không còn chỉ hỏi *"làm thế nào để dự đoán dữ liệu huấn luyện chính xác nhất?"* mà trở thành *"làm thế nào để vừa dự đoán tốt, vừa giữ cho mô hình không quá phức tạp?"*

Thông thường ta **không điều chuẩn hệ số chệch** $w_0$, vì $w_0$ chỉ đặt mức giá cơ sở và không làm mô hình "cong" hơn. Để viết điều đó bằng ma trận, ta đặt

$$\mathbf{P} = \text{diag}(0,1,1,\dots,1) = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

Khi đó $\mathbf{w}^T\mathbf{P}\mathbf{w} = \sum_{j=1}^{d} w_j^2$ và $\| \mathbf{P}\mathbf{w} \|_1 = \sum_{j=1}^{d} |w_j|$: cả hai đều bỏ qua $w_0$.

## Ridge: điều chuẩn L2

**Ridge** phạt **bình phương** các trọng số:

$$J_{\text{Ridge}}(\mathbf{w}) = \frac{1}{n} \| \mathbf{X}\mathbf{w} - \mathbf{y} \|^2 + \lambda \, \mathbf{w}^T\mathbf{P}\mathbf{w}$$

Gradient là tổng gradient của MSE và gradient của penalty:

$$\nabla J_{\text{Ridge}}(\mathbf{w}) = \frac{2}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda \, \mathbf{P}\mathbf{w}$$

Quy tắc cập nhật Gradient Descent có thể viết lại thành

$$\mathbf{w} \leftarrow (\mathbf{I} - 2\eta\lambda\mathbf{P})\mathbf{w} - \eta \, \frac{2}{n}\mathbf{X}^T\mathbf{e}$$

Số hạng đầu cho thấy mỗi bước, mọi trọng số $w_j$ ($j \geq 1$) bị **nhân với hệ số** $1 - 2\eta\lambda < 1$, tức là bị **co nhỏ** lại một chút trước khi nhận bước điều chỉnh từ dữ liệu. Vì vậy Ridge còn gọi là **weight decay** (suy giảm trọng số).

Ridge còn có nghiệm đóng. Cho gradient bằng $\mathbf{0}$ ta được

$$(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{P})\mathbf{w}^* = \mathbf{X}^T\mathbf{y}$$

Với $\lambda > 0$, ma trận $\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{P}$ **luôn khả nghịch**, kể cả khi các đặc trưng tương quan mạnh hoặc $n < d + 1$. Đây là lý do Ridge **ổn định hơn** phương trình chuẩn thông thường (mục 3.7).

**Ý nghĩa.** Ridge thêm chi phí cho các trọng số lớn: trọng số càng lớn, chi phí càng cao. Mô hình có xu hướng dùng các trọng số nhỏ hơn, nên **đơn giản và ổn định hơn**. Trên dữ liệu ở mục 4.3, Ridge với $\lambda = 0{,}01$ cho đa thức bậc 8 độ lớn trọng số chỉ $0{,}92$ thay vì $182\,752$, và MSE kiểm tra giảm từ $0{,}1221$ xuống $0{,}0543$ (Hình 8b).

## Lasso: điều chuẩn L1

**Lasso** phạt **tổng giá trị tuyệt đối** của các trọng số:

$$J_{\text{Lasso}}(\mathbf{w}) = \frac{1}{n} \| \mathbf{X}\mathbf{w} - \mathbf{y} \|^2 + \lambda \, \| \mathbf{P}\mathbf{w} \|_1$$

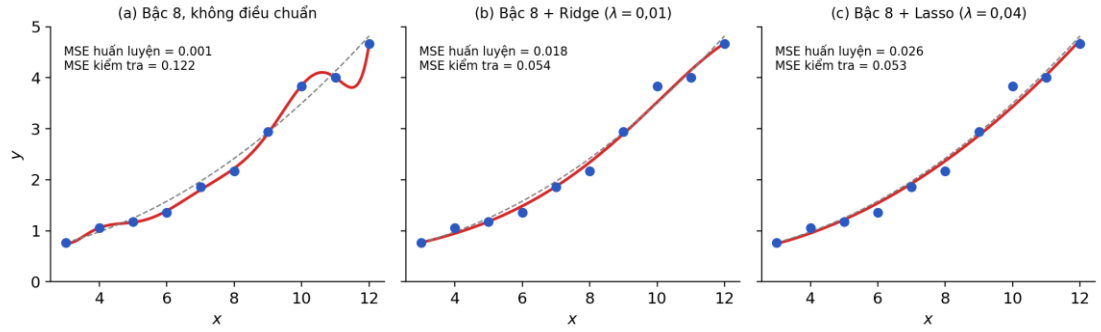
Vì $|w_j|$ không khả vi tại 0, ta dùng dưới đạo hàm. Với $w_j \neq 0$, $\frac{d|w_j|}{dw_j} = \text{sign}(w_j)$, còn tại $w_j = 0$ dưới đạo hàm là **cả đoạn** $[-1,1]$:

$$\partial|w_j| = \begin{cases} \{1\} & w_j > 0 \\ [-1,1] & w_j = 0 \\ \{-1\} & w_j < 0 \end{cases}$$

Do đó một dưới đạo hàm của hàm mục tiêu Lasso là

$$\partial J_{\text{Lasso}}(\mathbf{w}) = \frac{2}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda\,\mathbf{P}\text{sign}(\mathbf{w})$$

**Điểm đặc biệt của Lasso** là nó có xu hướng đưa một số trọng số về **đúng bằng** 0. Khi $w_j = 0$, đặc trưng tương ứng không còn đóng góp vào dự đoán, nên Lasso thực hiện một dạng **lựa chọn đặc trưng (feature selection)**. Trên dữ liệu ở mục 4.3, Lasso với $\lambda = 0{,}04$ giữ lại chỉ 2 trong 8 đặc trưng đa thức (chủ yếu là $x^2$, đúng với quan hệ thật), đưa 6 trọng số còn lại về 0 và đạt MSE kiểm tra 0,0527 (Hình 8c).




*Hình 8. Đa thức bậc 8 trên dữ liệu ở Hình 6: (a) không điều chuẩn, (b) Ridge, (c) Lasso. Điều chuẩn loại bỏ các dao động của mô hình quá khớp.*

**Vì sao L1 cho trọng số bằng 0 còn L2 chỉ co nhỏ?** Hãy nhìn "lực kéo" mà penalty tác động lên $w_j$. Ở Ridge, lực kéo là $2\lambda w_j$, **yếu dần** khi $w_j$ tiến về 0, nên $w_j$ nhỏ đi nhưng hiếm khi chạm đúng 0. Ở Lasso, lực kéo là $\lambda\text{sign}(w_j)$, **có độ lớn không đổi** bất kể $w_j$ lớn hay nhỏ, nên nó có thể kéo thẳng $w_j$ về đúng 0 nếu dữ liệu không đủ mạnh để chống lại.

Vì dưới đạo hàm của Lasso không liên tục tại 0, Gradient Descent thông thường không phù hợp. Lasso thường được huấn luyện bằng **Coordinate Descent** (tối ưu từng trọng số một) hoặc **Proximal Gradient**. Đây là ví dụ cho thấy **đổi hàm mục tiêu có thể buộc ta đổi cả thuật toán tối ưu**.

## Siêu tham số $\lambda$

Các trọng số $w_0, w_1, \ldots, w_d$ được **học từ dữ liệu**. Ngược lại, $\lambda$ là **siêu tham số (hyperparameter)** do người xây dựng mô hình chọn, thường thông qua tập kiểm chứng (validation) hoặc kiểm chứng chéo (cross-validation):

- $\lambda = 0$: không điều chuẩn, quay về MSE thuần túy;
- $\lambda$ nhỏ: điều chuẩn yếu, mô hình vẫn có thể quá phức tạp và quá khớp;

- $\lambda$ lớn: điều chuẩn mạnh. Khi $\lambda \to \infty$, mọi $w_j$ $(j \geq 1)$ tiến về 0 và mô hình luôn dự đoán giá trị trung bình $\bar{y}$, tức là **underfitting**.

Cần chọn $\lambda$ **vừa phải** để cân bằng giữa dự đoán tốt và mô hình đơn giản. Vì penalty phụ thuộc trực tiếp vào độ lớn của $w_j$, mà độ lớn này phụ thuộc thang đo của đặc trưng, ta **luôn chuẩn hóa đặc trưng trước khi dùng Ridge hoặc Lasso**.

## 5.3. Tổng hợp các hàm mục tiêu

Bảng sau tổng hợp bốn hàm mục tiêu đã học:

| Mô hình | Hàm mục tiêu $J(\mathbf{w})$ | Gradient hoặc dưới đạo hàm | Tối ưu |
|---|---|---|---|
| MSE | $\frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|^2$ | $\frac{2}{n}\mathbf{X}^T\mathbf{e}$ | Gradient Descent, phương trình chuẩn |
| MAE | $\frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|_1$ | $\frac{1}{n}\mathbf{X}^T\text{sign}(\mathbf{e})$ | Subgradient descent |
| Ridge | $\frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|^2 + \lambda \mathbf{w}^T\mathbf{Pw}$ | $\frac{2}{n}\mathbf{X}^T\mathbf{e} + 2\lambda\mathbf{Pw}$ | Gradient Descent, nghiệm đóng |
| Lasso | $\frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|^2 + \lambda \| \mathbf{Pw} \|_1$ | $\frac{2}{n}\mathbf{X}^T\mathbf{e} + \lambda\mathbf{P}\text{sign}(\mathbf{w})$ | Coordinate Descent, Proximal Gradient |

Ridge và Lasso **không phải là hai mô hình khác** với hồi quy tuyến tính. Về cấu trúc dự đoán, cả hai vẫn là mô hình tuyến tính $\hat{y} = \mathbf{xw}$. Điểm khác nằm ở **hàm mục tiêu được tối ưu**, tức là cách huấn luyện. Ta có thể tóm gọn:

$$\text{Hàm mục tiêu} = \text{Loss dữ liệu} + \text{Penalty độ phức tạp}$$

Từ đây ta có ba câu hỏi cơ bản để phân tích **bất kỳ** mô hình học máy nào, không chỉ hồi quy tuyến tính:

1. Mô hình có dạng gì? (Hồi quy tuyến tính: $\hat{y} = \mathbf{xw}$.)
2. Ta dùng hàm mục tiêu nào để đánh giá và kiểm soát mô hình? (MSE, MAE, Ridge, Lasso.)
3. Ta dùng thuật toán nào để tối ưu các tham số? (Gradient Descent, phương trình chuẩn, Coordinate Descent.)

Trả lời được ba câu hỏi này là nắm được **bộ khung cốt lõi của quá trình huấn luyện một mô hình học máy**, và ta sẽ dùng lại nó ở bài Perceptron.

## Tóm tắt mục 5

- **MSE** phạt sai số lớn rất mạnh nên nhạy với ngoại lệ. **MAE** $= \frac{1}{n} \| \mathbf{Xw} - \mathbf{y} \|_1$ phạt tuyến tính nên bền hơn. MSE dùng **độ lớn** sai số trong gradient, MAE chỉ dùng **dấu**.

- **Điều chuẩn** thêm một penalty vào hàm mục tiêu để kiểm soát độ phức tạp: $J = J_{\text{MSE}} + \lambda R(\mathbf{w})$, với ma trận $\mathbf{P} = \text{diag}(0,1,\dots,1)$ để không điều chuẩn $w_0$.
- **Ridge** (L2): $\lambda \mathbf{w}^T \mathbf{P} \mathbf{w}$, gradient thêm $2\lambda \mathbf{P}\mathbf{w}$, co nhỏ các trọng số, có nghiệm đóng $(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{P})\mathbf{w} = \mathbf{X}^T\mathbf{y}$ luôn khả nghịch.
- **Lasso** (L1): $\lambda \parallel \mathbf{P}\mathbf{w} \parallel_1$, đưa một số trọng số về đúng 0 (chọn đặc trưng), cần dưới đạo hàm và các thuật toán như Coordinate Descent.
- $\lambda$ là **siêu tham số** chọn bằng dữ liệu kiểm chứng. Cần chuẩn hóa đặc trưng trước khi điều chuẩn.
- Ba câu hỏi phân tích mọi mô hình: **dạng mô hình, hàm mục tiêu, thuật toán tối ưu**.

## Câu hỏi kiểm tra nhanh

1. Cho vector sai số $\mathbf{e} = [1 \quad -2 \quad 4]^T$. Tính MSE và MAE. Nếu sai số thứ ba đổi từ 4 thành 40 thì MSE và MAE thay đổi thế nào?
2. Tính $\text{sign}(\mathbf{e})$ cho $\mathbf{e} = [0{,}5 \quad -2 \quad 0 \quad 3]^T$ với quy ước $\text{sign}(0) = 0$.
3. Với $\mathbf{w} = [2 \quad 3 \quad -1]^T$, $\lambda = 0{,}1$ và $\mathbf{P} = \text{diag}(0,1,1)$, hãy tính phần gradient do penalty của Ridge $2\lambda\mathbf{P}\mathbf{w}$.
4. Với Ridge, hệ số co $1 - 2\eta\lambda$ bằng bao nhiêu khi $\eta = 0{,}1$ và $\lambda = 0{,}5$? Giải thích vì sao khi $\lambda \to \infty$ mô hình chỉ dự đoán giá trị trung bình $\bar{y}$.
5. Ridge và Lasso, phương pháp nào có thể đưa trọng số về đúng 0? Vì sao?
6. Vì sao ta không điều chuẩn hệ số chệch $w_0$ và vì sao phải chuẩn hóa đặc trưng trước khi dùng Ridge hoặc Lasso?

### Gợi ý đáp án

1. Với $\mathbf{e} = [1 \quad -2 \quad 4]^T$: MSE $= \frac{1+4+16}{3} = 7$ và MAE $= \frac{1+2+4}{3} = \frac{7}{3} \approx 2{,}33$. Khi 4 đổi thành 40: MSE $= \frac{1+4+1600}{3} = 535$ (tăng gần 76 lần) và MAE $= \frac{1+2+40}{3} \approx 14{,}33$ (tăng khoảng 6 lần). MSE nhạy với ngoại lệ hơn rất nhiều.
2. $\text{sign}(\mathbf{e}) = [1 \quad -1 \quad 0 \quad 1]^T$.
3. $\mathbf{P}\mathbf{w} = [0 \quad 3 \quad -1]^T$ nên $2\lambda\mathbf{P}\mathbf{w} = 0{,}2 \cdot [0 \quad 3 \quad -1]^T = [0 \quad 0{,}6 \quad -0{,}2]^T$. Thành phần đầu bằng 0 vì $w_0$ không bị điều chuẩn.
4. $1 - 2 \cdot 0{,}1 \cdot 0{,}5 = 0{,}9$, tức mỗi bước các trọng số bị co còn 90%. Khi $\lambda \to \infty$, penalty áp đảo nên mọi $w_j$ ($j \geq 1$) tiến về 0. Khi đó $\hat{y} = w_0$, và $w_0$ làm MSE nhỏ nhất khi bằng trung bình $\bar{y}$.
5. **Lasso.** Lực kéo của penalty L1 là $\lambda \text{sign}(w_j)$ có độ lớn không đổi nên có thể kéo $w_j$ về đúng 0. Lực kéo của Ridge là $2\lambda w_j$ yếu dần khi $w_j$ nhỏ, nên chỉ co nhỏ chứ hiếm khi bằng 0.
6. $w_0$ chỉ đặt mức giá cơ sở, không làm mô hình cong hơn, và điều chuẩn nó khiến kết quả phụ thuộc vào gốc đo của $y$. Penalty phụ thuộc trực tiếp vào độ lớn $w_j$, mà độ lớn này phụ thuộc thang đo của đặc trưng, nên nếu không chuẩn hóa, đặc trưng đo bằng đơn vị nhỏ sẽ bị phạt bất công so với đặc trưng đo bằng đơn vị lớn.

# 6. Cài đặt và đánh giá mô hình hồi quy tuyến tính

Ở các mục trước, ta đã xây dựng hồi quy tuyến tính bằng toán học và tính tay trên 5 căn nhà. Mục này biến lý thuyết thành **chương trình chạy được** và trả lời một câu hỏi quan trọng không kém việc huấn luyện: **mô hình học được tốt đến đâu trên dữ liệu mới?** Nội dung gồm ba phần:

- Cài đặt hồi quy tuyến tính **từ đầu** bằng Python (chỉ dùng NumPy) và kiểm tra rằng kết quả khớp với ví dụ tính tay ở mục 3.
- Huấn luyện trên một bộ dữ liệu lớn hơn, **chia dữ liệu thành tập huấn luyện và tập kiểm tra**, rồi đánh giá bằng MSE, RMSE, MAE và $R^2$.
- So sánh với `LinearRegression`, `Ridge`, `Lasso` và `SGDRegressor` của **scikit-learn**.

Toàn bộ mã trong mục này đã được chạy thật, và các kết quả in ra đều lấy từ lần chạy đó. Ta dùng hai thư viện: NumPy (tính toán ma trận) và scikit-learn (so sánh).

## Quy ước chuyển từ ký hiệu toán sang mã

Ta giữ nguyên quy ước ký hiệu của toàn bài: **chữ hoa in đậm** là ma trận, **chữ thường in đậm** là vector, $\mathbf{x}_i$ là vector hàng, $\mathbf{w}$ là vector cột. Trong NumPy, ma trận là mảng hai chiều, còn vector là mảng một chiều. Bảng sau cho biết mỗi công thức ứng với dòng mã nào:

| Ký hiệu toán | Trong mã Python |
|---|---|
| Ma trận đặc trưng $\mathbf{X}$ (kích thước $n \times (d+1)$, cột đầu bằng 1) | `Xb` (kết quả của hàm `them_bias`) |
| Vector tham số $\mathbf{w}$ (độ dài $d+1$) | `w` |
| Vector đặc trưng của mẫu $i$, $\mathbf{x}_i$ | `Xb[i]` |
| Dự đoán $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ | `Xb @ w` |
| Sai số $\mathbf{e} = \mathbf{X}\mathbf{w} - \mathbf{y}$ | `Xb @ w - y` |
| Mất mát $J = \frac{1}{n} \| \mathbf{e} \|^2$ | `np.mean(e ** 2)` |
| Gradient $\nabla J = \frac{2}{n}\mathbf{X}^T\mathbf{e}$ | `(2 / n) * Xb.T @ e` |
| Cập nhật $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$ | `w = w - eta * grad` |
| Phương trình chuẩn $\mathbf{X}^T\mathbf{X}\,\mathbf{w} = \mathbf{X}^T\mathbf{y}$ | `np.linalg.solve(Xb.T @ Xb, Xb.T @ y)` |

## 6.1. Cài đặt từ đầu bằng Python

### Các hàm cơ bản

Dữ liệu thô là ma trận $n \times d$. Ta thêm cột toàn số 1 ở đầu để được $\mathbf{X}$ kích thước $n \times (d+1)$. Các hàm sau cài đặt đúng các công thức ở mục 2 và mục 3:

```
import numpy as np

def them_bias(X):
```

```
    """Thêm cột 1 vào đầu ma trận đặc trưng: (n, d) -> (n, d+1)."""
    return np.hstack([np.ones((X.shape[0], 1)), X])

def du_doan(Xb, w):
    """Giá trị dự đoán y_hat = Xw."""
    return Xb @ w

def mse(Xb, y, w):
    """Hàm mất mát MSE: J(w) = (1/n) ||Xw - y||^2."""
    e = Xb @ w - y
    return np.mean(e ** 2)

def gradient(Xb, y, w):
    """Gradient của MSE: (2/n) X^T (Xw - y)."""
    n = len(y)
    return (2 / n) * Xb.T @ (Xb @ w - y)
```

Chú ý rằng toàn bộ tính toán là **các phép nhân ma trận**, không cần vòng lặp qua từng mẫu.

## Hai cách huấn luyện

Hàm *huan_luyen_gd* cài đặt thuật toán ở mục 3.4: khởi tạo $\mathbf{w} = \mathbf{0}$, lặp, cập nhật và dừng khi $J$ gần như không giảm. Hàm *phuong_trinh_chuan* cài đặt mục 3.7. Ta dùng *np.linalg.solve* để giải hệ thay vì tính nghịch đảo *np.linalg.inv*, vì cách này **chính xác và nhanh hơn**.

```
def huan_luyen_gd(Xb, y, eta=0.02, t_max=1000, eps=1e-12):
    """Gradient Descent toàn bộ dữ liệu cho hồi quy tuyến tính."""
    w = np.zeros(Xb.shape[1])                  # khởi tạo w = 0
    lich_su = [mse(Xb, y, w)]                  # J sau mỗi lần cập nhật
    for t in range(t_max):
        w = w - eta * gradient(Xb, y, w)       # cập nhật tham số
        lich_su.append(mse(Xb, y, w))
        if abs(lich_su[-2] - lich_su[-1]) < eps:   # điều kiện dừng
            break
    return w, lich_su

def phuong_trinh_chuan(Xb, y):
    """Nghiệm đóng: giải (X^T X) w = X^T y."""
    return np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
```

Hàm *huan_luyen_gd* trả về vector tham số cuối cùng *w* và danh sách *lich_su* chứa $J$ sau mỗi lần cập nhật để theo dõi quá trình học.

## Kiểm tra trên dữ liệu 5 căn nhà

Cách tốt nhất để biết cài đặt đúng là so với kết quả đã tính tay. Ta chạy trên đúng 5 căn nhà ở mục 1.1:

```
X_nha = np.array([[3], [4], [5], [6], [7]], dtype=float)
y_nha = np.array([1.2, 1.9, 2.4, 3.2, 3.3])
Xb_nha = them_bias(X_nha)
```

```
w_pt = phuong_trinh_chuan(Xb_nha, y_nha)
print("Phuong trinh chuan: w =", w_pt, "| J =", round(mse(Xb_nha, y_nha,
w_pt), 4))

w_gd, J = huan_luyen_gd(Xb_nha, y_nha, eta=0.02, t_max=3, eps=0)
print("GD sau 3 lan lap  : w =", np.round(w_gd, 5))
print("J qua cac lan lap :", np.round(J, 4))

w_gd, J = huan_luyen_gd(Xb_nha, y_nha, eta=0.02, t_max=1000, eps=0)
print("GD sau 1000 lan   : w =", np.round(w_gd, 4), "| J =", round(J[-1], 4))

# chuẩn hóa đặc trưng
x_tb, x_sd = X_nha.mean(axis=0), X_nha.std(axis=0)
Xs = them_bias((X_nha - x_tb) / x_sd)
w_s, J_s = huan_luyen_gd(Xs, y_nha, eta=0.3, t_max=1000)
print("Sau chuan hoa: w =", np.round(w_s, 4), "| so lan lap =", len(J_s) - 1,
"| J =", round(J_s[-1], 4))
w_goc = np.array([w_s[0] - w_s[1] * x_tb[0] / x_sd[0], w_s[1] / x_sd[0]])
print("Quy ve thang do goc: w =", np.round(w_goc, 4))
```

Kết quả in ra:

```
Phuong trinh chuan: w = [-0.35  0.55] | J = 0.023
GD sau 3 lan lap  : w = [0.08345 0.4703 ]
J qua cac lan lap : [6.388  0.1242 0.0382 0.0369]
GD sau 1000 lan   : w = [-0.3252  0.5454] | J = 0.023
Sau chuan hoa: w = [2.4    0.7778] | so lan lap = 17 | J = 0.023
Quy ve thang do goc: w = [-0.35  0.55]
```

Mọi thứ **khớp hoàn toàn với ví dụ tính tay**: phương trình chuẩn cho $\mathbf{w}^* = [-0{,}35 \quad 0{,}55]^T$ và $J = 0{,}023$; dãy mất mát $6{,}388 \to 0{,}1242 \to 0{,}0382$ trùng với hai lần lặp ở mục 3.5; Gradient Descent với đặc trưng chưa chuẩn hóa còn chậm sau 1000 lần lặp, còn sau chuẩn hóa chỉ cần 17 lần lặp và quy về thang đo gốc đúng bằng $\mathbf{w}^*$.

## 6.2. Huấn luyện trên dữ liệu lớn hơn và đánh giá mô hình

Với 5 mẫu, ta không thể nói gì về chất lượng mô hình trên dữ liệu mới. Mục này dùng một bộ dữ liệu lớn hơn và đánh giá một cách bài bản.

### Dữ liệu

Ta **mô phỏng** 300 căn nhà với hai đặc trưng: $x_1$ là diện tích ($m^2$) và $x_2$ là khoảng cách đến trung tâm ($km$). Giá được sinh theo quan hệ $y = 0{,}8 + 0{,}045x_1 - 0{,}08x_2 +$ nhiễu (nhiễu có độ lệch chuẩn 0,4 tỷ đồng). Vì ta biết quan hệ thật, ta có thể kiểm tra xem mô hình có tìm lại được các hệ số này không.

```
rng = np.random.default_rng(42)
n = 300
dien_tich = rng.uniform(30, 120, n)            # x1: m^2
```

```
khoang_cach = rng.uniform(0.5, 15, n)          # x2: km tới trung tâm
gia = 0.8 + 0.045 * dien_tich - 0.08 * khoang_cach + rng.normal(0, 0.4, n)
# y: tỷ đồng
X = np.column_stack([dien_tich, khoang_cach])
print("X:", X.shape, "| y:", gia.shape)
print("y trung binh = %.2f, do lech chuan = %.2f" % (gia.mean(), gia.std()))
```

```
X: (300, 2) | y: (300,)
y trung binh = 3.47, do lech chuan = 1.28
```

## Tập huấn luyện và tập kiểm tra

Ta chia ngẫu nhiên dữ liệu thành 80% để **huấn luyện** và 20% để **kiểm tra**. Tập kiểm tra đóng vai trò "dữ liệu mới": mô hình **không được nhìn thấy** nó khi huấn luyện, nên sai số trên đó phản ánh khả năng tổng quát hóa (mục 4.3). Các tham số chuẩn hóa (trung bình, độ lệch chuẩn) chỉ được ước lượng **trên tập huấn luyện** rồi áp dụng cho cả hai tập, để thông tin của tập kiểm tra không "rò rỉ" vào quá trình huấn luyện.

```
idx = rng.permutation(n)
n_tr = int(0.8 * n)
tr, te = idx[:n_tr], idx[n_tr:]
X_tr, y_tr, X_te, y_te = X[tr], gia[tr], X[te], gia[te]
print("So mau huan luyen:", len(tr), "| so mau kiem tra:", len(te))

tb, sd = X_tr.mean(axis=0), X_tr.std(axis=0)       # chỉ dùng tập huấn luyện
Xs_tr = them_bias((X_tr - tb) / sd)
Xs_te = them_bias((X_te - tb) / sd)
```

```
So mau huan luyen: 240 | so mau kiem tra: 60
```

## Huấn luyện bằng hai cách

```
w_gd, J = huan_luyen_gd(Xs_tr, y_tr, eta=0.1, t_max=5000, eps=1e-12)
w_pt = phuong_trinh_chuan(Xs_tr, y_tr)
print("GD: so lan lap =", len(J) - 1, "| w =", np.round(w_gd, 4))
print("PT chuan        | w =", np.round(w_pt, 4))
print("Sai khac toi da giua hai nghiem: %.2e" % np.max(np.abs(w_gd - w_pt)))

# Quy về thang đo gốc để diễn giải
w1_g = w_pt[1:] / sd
w0_g = w_pt[0] - np.sum(w_pt[1:] * tb / sd)
print("He so tren thang do goc: w0 = %.4f, w1 = %.4f, w2 = %.4f" % (w0_g,
*w1_g))
```

```
GD: so lan lap = 68 | w = [ 3.4679  1.1855 -0.3627]
PT chuan        | w = [ 3.4679  1.1855 -0.3627]
Sai khac toi da giua hai nghiem: 8.92e-07
He so tren thang do goc: w0 = 0.8157, w1 = 0.0452, w2 = -0.0871
```

Hai cách huấn luyện cho **cùng một nghiệm** (sai khác dưới $10^{-6}$). Khi quy về thang đo gốc, ta thu được $\hat{y} = 0{,}8157 + 0{,}0452x_1 - 0{,}0871x_2$, rất gần quan hệ thật $0{,}8 + 0{,}045x_1 - 0{,}08x_2$.

Mô hình đã **tìm lại được cấu trúc** của dữ liệu từ các mẫu có nhiễu. Ở thang chuẩn hóa, $w_0 = 3{,}4679$ chính là giá trung bình $\bar{y}$ của tập huấn luyện, như đã nhận xét ở mục 3.5.

## Các thước đo đánh giá

Với vector sai số **e** trên một tập dữ liệu có $n$ mẫu, ta dùng bốn thước đo:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \qquad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}$$

- **RMSE** có cùng đơn vị với $y$ (tỷ đồng), nên dễ diễn giải hơn MSE. Vì bình phương, RMSE nhạy với sai số lớn hơn MAE, nên RMSE luôn lớn hơn hoặc bằng MAE.
- **MAE** là sai số tuyệt đối trung bình, cũng cùng đơn vị với $y$.
- $R^2$ (hệ số xác định) cho biết **tỉ lệ phương sai của $y$ được mô hình giải thích**. $R^2 = 1$ là dự đoán hoàn hảo. $R^2 = 0$ là mô hình không tốt hơn việc luôn dự đoán trung bình $\bar{y}$. $R^2 < 0$ là còn tệ hơn việc đó.

```
def danh_gia(y, y_hat):
    e = y_hat - y
    mse_ = np.mean(e ** 2)
    mae_ = np.mean(np.abs(e))
    r2 = 1 - np.sum(e ** 2) / np.sum((y - y.mean()) ** 2)
    return mse_, np.sqrt(mse_), mae_, r2

for ten, Xm, ym in [("Huan luyen", Xs_tr, y_tr), ("Kiem tra  ", Xs_te,
y_te)]:
    m, r, a, r2 = danh_gia(ym, du_doan(Xm, w_pt))
    print("%s: MSE = %.4f | RMSE = %.4f | MAE = %.4f | R2 = %.4f" % (ten, m,
r, a, r2))

# Mô hình trivial: luôn dự đoán giá trị trung bình
print("Baseline (du doan y trung binh): R2 tren tap kiem tra = %.4f" %
danh_gia(y_te, np.full(len(y_te), y_tr.mean()))[3])
```

```
Huan luyen: MSE = 0.1732 | RMSE = 0.4162 | MAE = 0.3340 | R2 = 0.8958
Kiem tra  : MSE = 0.1526 | RMSE = 0.3907 | MAE = 0.2995 | R2 = 0.9008
Baseline (du doan y trung binh): R2 tren tap kiem tra = -0.0000
```

| Tập dữ liệu | MSE | RMSE (tỷ đồng) | MAE (tỷ đồng) | $R^2$ |
|---|---|---|---|---|
| Huấn luyện | 0,1732 | 0,4162 | 0,3340 | 0,8958 |
| Kiểm tra | 0,1526 | 0,3907 | 0,2995 | 0,9008 |

Ta rút ra ba nhận xét:

- Trên dữ liệu kiểm tra, mô hình sai trung bình khoảng 0,30 tỷ đồng (MAE) và $R^2 \approx 0{,}90$, tức giải thích khoảng 90% biến thiên của giá nhà, trong khi mô hình "luôn đoán trung bình" có $R^2 \approx 0$.

- Sai số huấn luyện và sai số kiểm tra **gần nhau** nên không có dấu hiệu quá khớp. Điều này hợp lý vì mô hình chỉ có 3 tham số trong khi có 240 mẫu huấn luyện. Sai số kiểm tra thấp hơn một chút là do ngẫu nhiên của 60 mẫu, không phải lỗi.
- RMSE (0,39) xấp xỉ độ lệch chuẩn của nhiễu (0,4): mô hình đã đạt gần mức sai số tối thiểu có thể, vì phần nhiễu là không thể dự đoán.

### Điều chuẩn Ridge cài từ đầu

Ta cài đặt Ridge theo nghiệm đóng $(\mathbf{X}^T\mathbf{X} + n\lambda\mathbf{P})\mathbf{w} = \mathbf{X}^T\mathbf{y}$ ở mục 5.2 và thử bốn giá trị $\lambda$:

```
def ridge(Xb, y, lam):
    n_, d1 = Xb.shape
    P = np.eye(d1); P[0, 0] = 0               # không điều chuẩn w0
    return np.linalg.solve(Xb.T @ Xb + n_ * lam * P, Xb.T @ y)

for lam in [0, 0.1, 1, 10]:
    w_r = ridge(Xs_tr, y_tr, lam)
    print("lambda = %-4s w = %s | MSE kiem tra = %.4f" % (lam, np.round(w_r,
4), danh_gia(y_te, du_doan(Xs_te, w_r))[0]))
```

```
lambda = 0    w = [ 3.4679  1.1855 -0.3627] | MSE kiem tra = 0.1526
lambda = 0.1  w = [ 3.4679  1.0758 -0.3241] | MSE kiem tra = 0.1625
lambda = 1    w = [ 3.4679  0.5872 -0.1645] | MSE kiem tra = 0.5034
lambda = 10   w = [ 3.4679  0.1061 -0.0274] | MSE kiem tra = 1.3018
```

Khi $\lambda$ tăng, các trọng số $w_1, w_2$ **co nhỏ dần** về 0 còn $w_0 = \bar{y}$ không đổi (vì không bị điều chuẩn). Ở đây MSE kiểm tra lại **tăng** theo $\lambda$. Điều này không mâu thuẫn với lý thuyết: dữ liệu này có 240 mẫu cho 3 tham số, mô hình đúng dạng, nên **không có quá khớp để chữa**, và điều chuẩn chỉ làm mô hình bị dưới khớp. Điều chuẩn phát huy tác dụng khi mô hình phức tạp so với dữ liệu, như ở Hình 8. Đây là lý do $\lambda$ phải được **chọn bằng dữ liệu kiểm chứng** thay vì chọn tùy ý.

## 6.3. So sánh với scikit-learn

Trong thực hành, ta thường dùng thư viện thay vì tự cài đặt. Đoạn mã sau huấn luyện bốn mô hình của scikit-learn trên **cùng dữ liệu** `X_tr`, `y_tr` ở mục 6.2. Mỗi mô hình được đặt trong một `Pipeline` có `StandardScaler`, để việc chuẩn hóa được ước lượng trên tập huấn luyện đúng như ta đã làm:

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

n_tr = len(y_tr)
lam = 0.1
mo_hinh = {
    "LinearRegression": LinearRegression(),
    "Ridge (lambda=0,1)": Ridge(alpha=n_tr * lam),        # alpha = n *
lambda
```

```
    "Lasso (lambda=0,1)": Lasso(alpha=lam / 2),          # alpha = lambda /
2
    "SGDRegressor": SGDRegressor(loss="squared_error", penalty=None,
learning_rate="constant",
                                 eta0=0.01, max_iter=2000, tol=1e-6,
random_state=0),
}
print("%-20s %-28s %-9s %-9s %-7s" % ("Mo hinh", "He so (thang do chuan
hoa)", "MSE", "MAE", "R2"))
for ten, m in mo_hinh.items():
    pipe = make_pipeline(StandardScaler(), m).fit(X_tr, y_tr)
    y_hat = pipe.predict(X_te)
    h = pipe[-1]
    print("%-20s b=%.3f w=%s %.4f    %.4f    %.4f" % (ten,
float(np.ravel(h.intercept_)[0]), np.round(h.coef_, 3),
          mean_squared_error(y_te, y_hat), mean_absolute_error(y_te, y_hat),
r2_score(y_te, y_hat)))
```

```
Mo hinh              He so (thang do chuan hoa)   MSE       MAE       R2
LinearRegression     b=3.468 w=[ 1.186 -0.363] 0.1526    0.2995    0.9008
Ridge (lambda=0,1)   b=3.468 w=[ 1.076 -0.324] 0.1625    0.3157    0.8944
Lasso (lambda=0,1)   b=3.468 w=[ 1.133 -0.31 ] 0.1587    0.3096    0.8968
SGDRegressor         b=3.484 w=[ 1.215 -0.378] 0.1539    0.3027    0.8999
```

## Những điểm cần biết

- ***LinearRegression*** cho đúng nghiệm phương trình chuẩn của ta ($b = 3{,}468$, $\mathbf{w} = [1{,}186 \quad -0{,}363]$, MSE kiểm tra 0,1526). Trong scikit-learn, hệ số chệch được lưu riêng trong *intercept_* (ký hiệu $b$ trong kết quả, tương ứng $w_0$ của ta) và các trọng số trong *coef_*.
- **Tham số điều chuẩn có quy ước khác.** *Ridge* tối thiểu hóa $\| \mathbf{y} - \mathbf{X}\mathbf{w} \|^2 + \alpha \| \mathbf{w} \|^2$ (không chia cho $n$), nên để tương ứng với $\lambda$ của bài giảng này phải đặt $\alpha = n\lambda$. *Lasso* tối thiểu hóa $\frac{1}{2n} \| \mathbf{y} - \mathbf{X}\mathbf{w} \|^2 + \alpha \| \mathbf{w} \|_1$ nên $\alpha = \lambda/2$. Với cách đổi này, *Ridge* cho $\mathbf{w} = [1{,}076 \quad -0{,}324]$, **trùng** với kết quả tự cài đặt ở mục 6.2 ($\lambda = 0{,}1$).
- ***Lasso* huấn luyện bằng Coordinate Descent**, đúng như đã nói ở mục 5.2. Với $\lambda = 0{,}1$ chưa có trọng số nào bằng 0 vì cả hai đặc trưng đều hữu ích. Khi tăng $\lambda$ đủ lớn, các trọng số sẽ lần lượt về 0.
- ***SGDRegressor*** dùng Stochastic Gradient Descent (mục 3.4) nên kết quả **gần nhưng không trùng** nghiệm chính xác ($b = 3{,}484$ thay vì 3,468): mỗi bước dùng gradient của một mẫu nên đường đi dao động quanh cực tiểu. Với dữ liệu nhỏ như thế này, nên dùng *LinearRegression*.
- Cả bốn mô hình có chất lượng xấp xỉ nhau ($R^2$ từ 0,894 đến 0,901), vì đây là bài toán đơn giản không có quá khớp.

## Tóm tắt mục 6

- Mọi công thức đều cài đặt bằng **phép nhân ma trận**: *Xb @ w* cho dự đoán, *(2/n) * Xb.T @ e* cho gradient, *np.linalg.solve* cho phương trình chuẩn.

- Gradient Descent và phương trình chuẩn cho **cùng một nghiệm**. Chuẩn hóa đặc trưng giúp Gradient Descent hội tụ nhanh hơn, và nghiệm trên thang chuẩn hóa có thể quy về thang gốc để diễn giải.
- Luôn **chia dữ liệu thành tập huấn luyện và tập kiểm tra**, ước lượng tham số chuẩn hóa chỉ trên tập huấn luyện, và đánh giá trên tập kiểm tra.
- Các thước đo: **MSE, RMSE, MAE** (càng nhỏ càng tốt) và $R^2$ (càng gần 1 càng tốt, $R^2 \approx 0$ là không hơn việc đoán trung bình).
- Với scikit-learn, cần lưu ý quy ước tham số điều chuẩn: $\alpha_{\text{Ridge}} = n\lambda$ và $\alpha_{\text{Lasso}} = \lambda/2$.

## Câu hỏi kiểm tra nhanh

1. Một mô hình có $R^2 = 0$ trên tập kiểm tra nghĩa là gì? $R^2 < 0$ nghĩa là gì?
2. Vì sao tham số chuẩn hóa (trung bình, độ lệch chuẩn) phải ước lượng trên tập huấn luyện chứ không phải trên toàn bộ dữ liệu?
3. Trên tập kiểm tra, mô hình có RMSE $= 0{,}39$ và MAE $= 0{,}30$ (tỷ đồng). Hãy diễn giải hai con số này. Vì sao RMSE lớn hơn MAE?
4. Muốn dùng $\lambda = 0{,}02$ với $n = 240$ mẫu, cần đặt *alpha* bằng bao nhiêu cho `Ridge` và cho `Lasso` của scikit-learn?
5. Vì sao nên dùng `np.linalg.solve(Xb.T @ Xb, Xb.T @ y)` thay vì `np.linalg.inv(Xb.T @ Xb) @ Xb.T @ y`?

### Gợi ý đáp án

1. $R^2 = 0$ nghĩa là mô hình **không tốt hơn** việc luôn dự đoán giá trị trung bình $\bar{y}$. $R^2 < 0$ nghĩa là mô hình còn **tệ hơn** việc đó.
2. Nếu dùng toàn bộ dữ liệu, thông tin của tập kiểm tra bị "rò rỉ" vào quá trình huấn luyện, và sai số kiểm tra sẽ lạc quan hơn thực tế. Tập kiểm tra phải mô phỏng dữ liệu hoàn toàn mới.
3. Mô hình dự đoán sai trung bình khoảng 0,30 tỷ đồng (MAE), và RMSE 0,39 tỷ đồng cho thấy có một số sai số lớn hơn mức trung bình. RMSE bình phương sai số trước khi lấy trung bình nên nhạy với sai số lớn hơn, do đó luôn ≥ MAE.
4. `Ridge`: $\alpha = n\lambda = 240 \cdot 0{,}02 = 4{,}8$. `Lasso`: $\alpha = \lambda/2 = 0{,}01$.
5. Tính nghịch đảo tường minh tốn kém hơn và dễ gây sai số số học khi $\mathbf{X}^T\mathbf{X}$ gần suy biến. `solve` giải trực tiếp hệ phương trình nên **chính xác và nhanh hơn**.

# Tổng kết và kết nối với bài Perceptron

Bài giảng đi qua một mạch xuyên suốt: ta bắt đầu từ **bài toán** dự đoán số (mục 1), xây dựng **mô hình** $\hat{y} = \mathbf{xw}$ (mục 2), **huấn luyện** nó bằng hàm mất mát MSE cùng Gradient Descent hoặc phương trình chuẩn (mục 3), rồi **mở rộng** từng thành phần: đổi cách biểu diễn đặc trưng (mục 4), đổi hàm mục tiêu để chống ngoại lệ và kiểm soát độ phức tạp (mục 5), và cuối cùng **cài đặt, đánh giá** (mục 6).

| Thành phần | Hồi quy tuyến tính | Các mở rộng trong bài |
|---|---|---|
| **Mô hình** | $\hat{y} = \mathbf{xw}$ | Biến đổi đặc trưng: $\hat{y} = \boldsymbol{\phi}(\mathbf{x})\mathbf{w}$ |

| Thành phần | Hồi quy tuyến tính | Các mở rộng trong bài |
|---|---|---|
| **Hàm mất mát** | MSE | MAE, Ridge, Lasso |
| **Thuật toán tối ưu** | Gradient Descent, phương trình chuẩn | SGD, Mini-batch, Coordinate Descent |

**Cầu nối sang Perceptron.** Bài toán tiếp theo là **phân loại**: đầu ra không phải một số thực mà là một nhãn $y \in \{-1, +1\}$. Ta sẽ không bỏ đi những gì đã học. Biểu thức $s = \mathbf{xw}$ vẫn được giữ nguyên, nhưng được dùng để cho biết điểm nằm ở phía nào của một siêu phẳng. Có thể nhớ gọn:

$$\text{Perceptron} = \text{hồi quy tuyến tính} + \text{hàm dấu}$$

Hai thành phần cần thay đổi là **đầu ra của mô hình** (thêm hàm dấu) và **hàm mất mát** (MSE đo khoảng cách tới giá trị, không đo việc đúng hay sai nhãn). Khung *mô hình, hàm mất mát, thuật toán tối ưu* và các ký hiệu $\mathbf{x}_i$, $\mathbf{X}$, $\mathbf{w}$, $\eta$ được dùng lại nguyên vẹn.